Don’t Worry We Don’t Byte Final Group Submission

Member	StudentID	Github name
Ian Kligman	924122742	ikligman
Joe Bowen	922966782	josephb333
Cheng-Chieh Yao	923978442	Jamesyao0818
Enrique Cabral-Mixco	924056104	rommix408

https://github.com/CSC415-2025-Fall/csc415-filesystem-josephb333

Table of Contents:

- Plan / Approach for each phase pgs 2 - 3
- Description of File System pgs 4 - 7
- Shell Commands Table pg 8
- Limitations pg 9
- Issues / Resolutions
  - MS 1 pgs 10 - 12
  - MS 2 pgs 13 - 27
  - MS 3 pg 27 - 40
- Details of functions
  - MS 1 pgs 41- 56
  - MS 2 pgs 57 - 79
  - MS 3 pgs 80 - 90
- Tables For Responsibilities / Contributions
  - MS 1 pg 91
  - MS 2 pg 92
  - MS 3 pg 93
- Meetings Tables pgs 94 - 98
- Compilation & Initial Run pg 99
- Screenshots of Commands
  - Tests with Empty Files pgs 100 - 105
  - Tests with Contents pgs 106 - 110

MS 1 Plan / Approach/ Changes

We initially divided responsibilities by the three different structures necessary e.g. Enrique does VCB, Ian does FSM, and Joe and James do DE. We planned to have all our individual work done going into Friday meetings and then work on debugging/integrating all individual parts over the weekend.

- For MS 1 nothing changed that much, everything went generally smooth with the exception of a last minute discovery of a bug which Joe had to remedy by himself at the last minute as everyone was unavailable by the time it was discovered the day of submission. The decision was made to increase the frequency of meetings to better avoid that instance happening in future.

MS 2 Plan / Approach / Changes

Again after milestone one, newly introduced preventative measures of increased meeting frequency were put in place to prevent further instances of any one person getting stuck fixing stuff they weren’t responsible for. However, going into the first Friday meeting the team expressed a shared sentiment regarding a need to divide work more fairly in addition to them. We collaboratively decided to give Joe less work as a thank you. James was given more stuff to do as we felt his contributions were slightly lacking.

For a variety of reasons this plan ended up being non-viable. James could not keep up with the timeline and expressed a desire to give some of the functions back. Joe took them up. Generally everything else went much smoother than MS 1 with its emergency at the end.

Beyond the explicit changes of responsibilities, there were miscommunications regarding the need of functions given certain timelines and sometimes others ended up NEEDING to do what someone else was supposed to so they could verify their function worked. After a talk with the professor regarding the best remedy to this. The decision was made to stick to what each person was responsible for and attempt communication until it was no longer an option.

Furthermore, MS 2 had a major pivot point for us. We had planned on moving to discontiguous allocation. However, MS 3 James had never been able to correctly implement our intended Linked List

discontiguous allocation. For that reason moving forward the call was made to permanently shift to contiguous.

MS 3 Plan / Approach / Changes

Given the fact it seemed we were ahead of most other groups and had been given a two week extension a desire to ease off the gas was expressed. Combined with the fact that some members of our group started to go through deeply personal stuff we made the call to try and take a break / slow down.

We had that Tuesday off for Veteran’s Day, normally Tuesday’s we decided responsibilities after class, so we held off till Thursday. We didn’t want to put pressure on members who needed more time. So given the fact that the IO layer was similar to assignment 5; we gave the entire IO layer to Enrique, two of the 6 shell commands to James, and Ian and Joe were to handle the remaining 4 together. This was so that come Friday's meeting we could at least have made some progress.

That plan went immediately out the window as Ian and Joe realized that unlike MS 2 where there were a proportion of MFS functions inherently responsible for shell commands; instead for MS3, effectively ALL of them were done via the IO layers functionality.

Because of this the call was made to just have everyone collaborate on the IO layer together. James was not at this meeting though so we were going to collaborate between Joe, Ian, Enrique on the IO layer and see how much we had left to give James once he was available.

By the time James came back to meetings on Monday we had basically finished MS 3 as the IO layer was done. We didn’t want James to have no way to contribute so we agreed that he could try and help us catch up on properly formatting the header in all files, removing old comments/artifacts, and help us do the descriptions of ALL individual functions for the writeup that we had been lagging on doing since MS 1.

However, James had told us that it would take him two weeks to do. Which the rest of the group agreed to be way too long. So the call was made to revert to the original plan for everyone to be responsible for the descriptions of functions they wrote.

Volume Control Block Description

The VCB struct itself is formatted such that it holds all the necessary information we need.

```
// Volume Control Block structure
// Stored at block 0 on disk
struct VCB
    {
    unsigned int numBlocks;         // Total number of blocks in the volume
    unsigned int blockSize;         // Block size in bytes
    unsigned int blockIndexFSM;     // Index of the block containing the File System Metadata
    unsigned int blockIndexRootDirectory;  // Index of the block containing the Root Directory
    unsigned int MagicNo;           // Magic number identifier (signature)
    // For Preallocation Strategy
    unsigned int numFreeBlocks;     // Number of blocks outside of LBA 7
    unsigned int freeTail;          // Highest lba of last write to FSM.
    };
```

Within it some values are kept for ease of consistency and ease of access cross modules. E.g. fsInit.c, DE.c

- numBlocks
- blockSize
- blockIndexFSM
- blockIndexRootDirectory
- MagicNo

Some are maintained and used through processes to facilitate functionality across all modules. E.g. FSM, MFS, etc.

- numFreeBlocks
- freeTail

The only additional necessary functions include an initialize function, a way to write/read the VCB to/from disk, and a fetcher function for use in initFSM().

```
// Initialize a new VCB and write it to disk (for formatting new
// Returns 0 on success, -1 on error
int initVCB(uint64_t numberOfBlocks, uint64_t blockSize);

// Read VCB from disk (block 1) into memory
// blockSize is needed to allocate the correct buffer size
// Returns 0 on success, -1 on error
int readVCB(uint64_t blockSize);

// Write VCB from memory to disk (block 1)
// Returns 0 on success, -1 on error
int writeVCB(void);

// Get pointer to in-memory VCB (for other modules to access)
// Returns NULL if VCB not loaded
struct VCB* getVCB(void);
```

Free Space Map Description

We decided to use a bitmap to keep track of free space. The first functions to implement were getBit, setBit, and clearBit so that they can be utilized by the initialize and allocate functions. These functions are static because they are meant to only be called by initFSM and allocate. They use bitwise logic operations with bitmasks to single out 1 bit.

The fsm char * will be maintained in memory as a global static variable. That way, each allocate call does not involve reading/writing to the disk. To keep the in memory version up to date with disk, callers can use fsmWrite after making changes (such as repeated calls to allocate) which writes the entire 5 blocks of fsm to disk.

The initFSM function is what mallocs the global static fsm, sets all blocks to 0 (free), then marks block 1 as used (VCB) and blocks 2-6 as used (FSM), marks block 7 as used (Root directory), and writes the map to disk.

The loadFSM function allows the initFS function to simply load a pre-existing fsm from disk into memory in the event that a matching signature is detected on startup. This is also important during an initial formatting of a volume, as the init functions all get called first (including initFSM, which writes to disk). However after that, the system will attempt to load that newly initialized fsm into memory.

```
//Function Declarations.
void fsmWrite();
void allocate(int tailLocation);
int initFSM(int blockSize);
static void setBit(int bitToSet);
static void clearBit(int bitToClear);
static int getBit(int bitToGet);
int fsmReserveLBA(int lba);
int fsmReleaseLBA(int lba);
int fsmIsUsed(int lba);
int loadFSM (int blockSize, unsigned int startBlock);
```

It also contains accessor functions fsmReserveLBA(), fsmReleaseLBA(), and fsmIsUsed() for the purposes of allowing other metadata layers maintain FSM validity as needed.

Director Entries Description

Here is the visual reference of the Directory Entry structure.

```c
//Directory Entry Struct
typedef struct DE {
    // Starting head of the file's location (for discontiguous files)
    int blockNumber;

    // File name
    char name[32];

    // File size in bytes
    unsigned int numBytes;

    // Flag to distinguish directory vs file
    int isDir;  // 1 for directory, 0 for file

    // Relevant Dates and Times (using time_t instead of char arrays)
    time_t createDate;
    time_t modifyDate;
    time_t accessDate;

} DE;
```

Directory Entries contain all the necessary meta data for access, book-keeping, read/writing to disk, flags to indicate whether dir/file.

IO Layer Description

This is the lowest-layer abstraction of components. It handles the I/O’s (in’s and outs) of data writes/reads/opens/closes. It does so by using properly buffered read/write loops, making sure to update the upper-level metadata of MFS, VCB, FSM layers in close when it is done.

fsInit Layer Description

Is where we handle the booting up for our filesystem with two main branches for whether it is starting fresh (no volume) or if volume exists i.e. VCB found with matching signature. It also contains the function for safe exit making sure to write VCB and FSM to disk when turned off so the FS’s state persists between runs.

Master File System Layer Desc.

As the name would suggest this is where the majority of our FS’s functionality exists. It is where all other layers interact to talk/change/sync with each other while performing functionality as the user prompts the shell.

Shell Layer Description

The shell is not only the driver for our entire FS but it is also the interface between it and the user. Under the hood it is calling all of the functions we implemented to perform functionality inherent to a file system by responses given by the user in the REPL.

Shell Commands Table

ls	Works as expected
cp	Works as expected
mv	Works as expected
md	Works as expected
touch	Works as expected
cat	Works as expected
cp2l	Works as expected
cd	Works as expected
pwd	Works as expected

Limitations

Although we were able to get all of the functionality needed for the project done there are some lingering fundamental limitations to our design as well as fixes we could have done given sufficient time/need outside of the scope of this project.

Our Volume is a fuse.

Because of our switch to contiguous files at such a late point. We had to quickly shift a lot of the components. Given that we still came up with some pretty crafty ways to improve functionality of how we handled external fragmentation and running out of space.

We made a tail in the VCB to keep track of where the last allocation was in FSM + 1. That way we know exactly which block is the starting free block all the way to the end of the volume. In addition, we came up with some (albeit uncommon) conditions where we could bring it back given we deleted something we just made, giving all those blocks back and moving the tail with it.

But eventually the tail is going to reach the end and if we did any removes in between creates there will be external fragmentation. We did not implement a way to handle what happens when we reach the end but we had to conceptual solutions one minor the other a sufficient “fix” but quite inefficient and hard to implement. They are as follows.

1. When we reach the end of the volume, we scan through from the beginning of the volume looking for the first best fit to allocate a file we want to create. By disregarding the tail we could start again and given 1 block files we could hypothetically always be able to fill the volume.
   - Totally within the scope of our potential however, we felt it was unnecessary.
2. If we reached end of volume as soon as we do, we do a depth first search to find the further branch from root, and then do a bunch of backtracking until we create a full copy of all DE’s, we can then go back and rewrite every single one making sure to not leave any gaps in between and get rid of any external fragmentation up until end of volume.
   - Again this is less feasible but conceptually possible.

Both of these issues and their suboptimal solutions reflect difficulties intrinsic to contiguous allocation. Which is partially why for the scope of this project we are okay with them.

Issues & Resolutions MS 1

We identified several critical issues in the original Directory Entry implementation that needed to be addressed to create a functional file system. The issues focused on five main areas: date storage, volume control structure, freespace management, entry type identification, and discontiguous block tracking.

The first issue was the date storage implementation. We decided to use time_t instead of character arrays to store dates. The original implementation used char arrays like createDate[] to store dates in DD/MM/YYYY format, which is format-dependent and difficult to manipulate. time_t is a standard type that stores dates as seconds since the Unix epoch, making date comparison and arithmetic much simpler. These date fields are now stored as time_t values. This approach is standard because it allows easy use of functions like time(), difftime(), and localtime() from the standard library, and eliminates format-dependency issues.

The second issue involved adding a VCB structure. We decided to implement a VCB to maintain file system metadata. The final rendition of the VCB structure contains essential information including number of blocks, block size, Logical Block Array LBA index of FSM, and magic no which is our signature.

```
// Volume Control Block structure
// Stored at block 0 on disk
struct VCB
    {
    unsigned int numBlocks;         // Total number of blocks in the volume
    unsigned int blockSize;         // Block size in bytes
    unsigned int blockIndexFSM;     // Index of the block containing the File System Metadata
    unsigned int blockIndexRootDirectory;  // Index of the block containing the Root Directory
    unsigned int MagicNo;           // Magic number identifier (signature)
    };
```

To distinguish between directories and files, we decided to add an isDir flag to the Directory Entry structure. The original DE structure had no way to identify whether an entry represented a file or a directory, which is essential for proper file system operations. The isDir field is an integer that is set to 1 for directories and 0 for regular files. This flag allows the system to perform appropriate operations based on entry type - directories can be traversed and listed, while files can be read and written. Without this field, operations like recursive

directory traversal would be impossible, and the system could not prevent invalid operations like trying to read a directory as if it were a file.

At first, we ran into the issue of not accounting for the professor’s partition table occupying physical block 0. The FSM needs to account for this offset, because it will assign blocks to those who call allocate. If it is off by one, the free space map will assign already used blocks to the caller! To correct this, we made sure to mark block 0 as used in the fsmInit. This way “our” block 0, where we understand the VCB to be assigned, will correctly be assigned to physical block 1 and so on.

The next issue was initially writing a needlessly complicated implementation of allocate(). This first version returned the starting block number of a group of n free blocks as requested by the caller. However,for our discontiguous allocation files, we only need to allocate to serve one free block at a time, regardless of whether or not these free blocks are next to each other. This first implementation would only be necessary if we were using contiguous files. To fix this we simplified the allocate() function to simply return one-at-a-time free blocks.

The first issue I ran into involved the two of us (James and I) working on the same part (Root Directory). We were having trouble coordinating so we both ended up making different changes to DE struct and methods which made it confusing to iterate through.

I resolved this by utilizing the steps that Professor had posted and going through his code step by step to make sure that it was doing what it was correctly and that it wasn’t missing any parts. By having a structured lens through which I could reference the work being done it made it way easier to iterate upon his work.

The next issue I had was about the non-functional entanglement between the three pieces (VCB, FSM, and Root Dir). During our third meeting our goal was to get all of our individual parts to work such that we could get a visible hex dump.

I resolved it by realizing there had been confusion surrounding what parts should be where. For instance, there had been declarations (with no implementation) of functions that were in the FSM (Free Space Map) that had leaked into DirectoryEntry.c. Part of the issue solution involved #including

the correct files in their respective places. Once we sorted out what needed what and where we finally got it to compile.

The next issue we faced was a deluge of warnings/errors we were getting once we compiled.

I resolved the issue by coordinating with each part’s respective parts and utilizing line itemized error feedback from the terminal to find, identify, and remediate errors one by one. After fixing about 20. We were finally about to get a fully compiled program with no errors.

The next issue we faced was the fact that our hexdump was displaying info contrary to what we expected it to do.

We were able to resolve it by carefully reading through the hexdump module and realizing that the Professor had a partition table at physical block 0. Once we realized that we displayed blocks 1, 2, and 7 respectively to see if our data was working correctly.

The next issue we faced was the fact that although we were able to see the data we wanted with the hex dumps, we realized that they were in the incorrect blocks. E.g. Block 7 which we expected to have the root directory was empty and was at block 8.

We resolved it by realizing that something was going wrong with the changes we made to account for physical block 0 being reserved for the partition table. Our logic to account for it throughout was inconsistent and unrobust. We had to go through all previous iterations to code to make sure that we updated all the necessary writes/reads to make sure they were looking at correct blocks. Also we had to fix the fact that the FSM was missing load functionality. The reason this was critical was because we had set up initFSM() such that it had two paths: 1. If the signature isn’t recognized, we format the volume. 2. That it is and we simply load VCB and FSM. We were loading the FSM() in initFSM() but in the case that the file system wasn’t recognized it was not. As such the allocate() function we used in the FSM wouldn’t be able to give the correct available block number. Furthermore, we had previous functional remnants of code that still did not account for +1 translation between physical and logical block addresses. Also we realized that LBAwrite() correctly accounted for physical address input. So all we had to do was change all lbawrites() back to their physical address, and include a more robust path for a found file system and then finally our blocks were showing up in their correct places.

Issues & Resolutions MS 2

mk_dir

One issue was the order in which writeDirToDisk was being called at the end of the function. The mistake was that the parent was being written before the new child directory, and that was then being overwritten. The fix was to simply switch the positions of the two writeDirToDisk calls so that the new dir is written first, then the parent is updated to reflect its new child.

Another issue was the FSM not being updated on each mk_dir write, and therefore the function allocate(), which marks and returns the next free blocks, was assigning already occupied block numbers to other callers! This caused many blocks to be overwritten and clobbering each other. The fix was to keep track of which block belongs to the child/parent using the functionality that we had already implemented in the free space map.

fs_stat

The issue when writing the function is that when writing the file size, we didn’t notice and forgot that we needed to follow and use the size from VCB. So how to fix that is to go to see how big the size has been defined or set for the system, and then come back to fs_stat to have the exactly the same size to use in the system.

loadDirectory()

This function, although it seemed simple at first, was at the root of a lot of confusion and subsequent issues we faced in MS 2. Fundamentally we made a mistake in MS 1 thinking that our FSM was holding LBA’s which were a +1 conversion from PBA due to the partition table being at physBlock 0. Throughout the many renditions of us trying to get the conversion logically consistent everywhere I found out the main issue here in loadDirectory().

```
// Convert from logical block index to physical
// Double check this part
int physBlock = dirEntry->blockNumber;
printf("In loadDirectory(): lbaReading block number : %d\n", physBlock);
// Make sure we also handle discontiguous instances. Pending James's Linked List implementation.
u_int64_t blocksRead = LBAread(buffer, blocksToRead, physBlock);
```

Here you can see why it finally clicked for me eventually. We were converting from PBA to LBA but then using that to feed to **LBA**read(). I finally took a pause to think if our current conversions were necessary. After changing everything to reflect LBA’s, adding debug statements to track, and using the Hex Dump to verify. I realized we had been doing unnecessary conversions the whole time and doing so inconsistently. I resolved this innate flaw in loadDirectory (and really everywhere else in our system) to maintain, use, and track LBA’s only. Which fixed the issue.

fs_isFile()

```
// Joe
// Returns 1 if file, 0 for dir, and -2 if called with bad parameters.
int fs_isFile(char * filename){
```

fs_isFile() unlike other functions this function is passed in a filename and not a path name. The real problem that I faced was in conceptualizing how I was going to verify filename’s is.Dir flag. In all other functions we needed to use parsePath() and pathname passed in to figure out what DE we are even referencing.

```
// Returns 1 if file, 0 otherwise;
int fs_isFile(char * filename){
    if (!filename || strlen(filename) == 0) return 0;
    parsePathInfo ppi;
    if (parsePath(filename, &ppi) != 0);
    printf("Error in fs_isFile: Invalid path\n");
    int index = findInDir(ppi.parent, filename);
    if (ppi.parent[index].isDir != 1) return 1;
    return 0;
}
```

Here you can see in the first rendition I was using it in our first draft.

However because this was a filename I faced quite a conceptual hang up and was stuck for a while.

fs_isFile()continued...

I resolved this confusion by realizing that if we are only given a filename we can assume that we only ever need to look for the file’s isDir flag in our CWD. Given that, I could just make a pointer to our cached global for CWD. Look through the number of its entries and if I find a matching name check its isDir flag.

```
// Joe
// Returns 1 if file, 0 for dir, and -2 if called with bad parameters.
int fs_isFile(char * filename){
    if (strcmp(filename, ".") == 0) return 0;
    if (strcmp(filename, "..") == 0) return 0;
    if (!filename || strlen(filename) == 0) return -2;
    printf("File name in isFile():%s\n", filename);
    DE *cwd = alreadyLoadedCWD;
    int index = findInDirectory(cwd, filename);
    if (index < 0) return -2;
    if (cwd[index].isDir == 1)
        return 0;
    return 1;
}
```

This improvement was not only simpler, easier to read, but also faster. Because now we didn’t need to parse the whole path, significantly optimizing the approach and decreasing likelihood of errors.

fs_setcwd()

```
+ //linux chdir
+ int fs_setcwd(char *pathname){
+     if (!pathname || strlen(pathname) == 0) return 0;
+     parsePathInfo ppi;
+     if (parsePath(pathname, &ppi) != 0);
+     printf("Error in fs_setcwd: Invalid path\n");
+     if (fs_isFile(ppi.lastElementName) == 0)
+     alreadyLoadedCWD = ppi.parent;
+ }
```

Here is our first draft of fs_setcwd()
You should very quickly notice quite a bit of problems in its logic.

COMPARE TO FINAL DRAFT ON NEXT PAGE FOR VISUAL AID TO BETTER UNDERSTAND ALL THE FOLLOWING ISSUES.

`fs_setcwd()`

```c
// CD - Parse pathname passed in by caller, checks for bad call
// e.g. pathname is invalid, pathname points to non-dir, uses that
// to update global cwd cache.
int fs_setcwd(char *pathname){
    //printf("Path name prior in setCwd():%s\n", pathname);
    //char * cwdPath = fs_getcwd(pathname, sizeof(pathname));
    //printf("Path name of cwd setCwd():%s\n", cwdPath);
    if (!pathname || strlen(pathname) == 0) return 0;
    DE *child = NULL;
    parsePathInfo ppi;
    // Populate ppi struct
    if (parsePath(pathname, &ppi) != 0) {
        printf("Error in fs_setcwd: Invalid path\n");
        return -1;
    }
    // Case : Path refers to parent (eg. '/')
    if (!ppi.lastElementName){
        //path already points to cwd
        alreadyLoadedCWD = ppi.parent;
        return 0;
    }
    // Assign local scope index from ppi struct for safety.
    int index = ppi.index;
    if (index < 0){
        printf("In fs_setcwd: could not resolve lastElementName.\n");
        return -1;
    }
    // Make sure we caller isn't trying to change to non-directory.
    if (ppi.parent[index].isDir != 1){
        printf("fs_setcwd: %s is not a directory.\n", ppi.lastElementName);
        return -1;
    }
    // Case : Final element is dir and exists.
    child = &ppi.parent[index];
    // Load directory metadata off disk.
    DE *newCWD = loadDirectory(child);
    //printf("DEBUG cd: found '%s' at idx=%d, isDir=%d, startLBA=%d size=%d\n",
       //ppi.lastElementName, index, child->isDir,
       //child->blockNumber, child->numBytes);
    if (!newCWD){
        printf("fs_setcwd: failed to load directory '%s'\n", ppi.parent[index].name);
        return -1;
    }
    // Free parent only if parsePath() loaded it dynamically from global cache's.
    if (ppi.parent != alreadyLoadedRootDir && ppi.parent != alreadyLoadedCWD){
        free(ppi.parent);
    }
    // Assign global cache to correct directory.
    alreadyLoadedCWD = newCWD;
    return 0;
}
```

In our first draft we not only severely underestimated the logic necessary to avoid a deluge of our subsequent errors but ALSO the nesting of fs_isFile() and fs_isDir() in their original states CALLING PARSEPATH AS WELL caused quite a headache for us. For simplicity let’s address each of these issues one at a time.

fs_setcwd()

1. Diagnosing what sort of mutation of pathname is occurring.

So initially I figured, “MY LOGIC MUST BE CORRECT LET ME SEE WHERE OTHER THINGS ARE GOING WRONG”. So obviously in my ignorance, I went to diagnose what issue was in parsePath and other relevant functions.

printf(“In fs_setcwd() path name is: %s”, pathname)

A seemingly inconsequential debug statement led me down a rabbit hole of doing a whole bunch of useless work and kept me from addressing the real issues.

Again it is right here and maybe it will be easy to notice the albeit small mistake. But I had put a space BEFORE the format specifier. Obviously seeing the output of:

In fs_setcwd() path name is: dir

I thought, “Oh the Professor thinks he is so tricky, somehow under the hood pathname takes in the space between argvector components. Oh now I gotta make a function to trim the string if leading a trailing whitespace. Annoying but easy enough. I did this in my first programming class. That makes sense as to why everything else isn’t working.”

```
char *trimString(char *pathname){
    if (!pathname) return NULL;
    // Trim from left when you see whitespace.
    while (*pathname && isspace((unsigned char)*pathname))
    pathname++;
    // Create and buffer string to return adding bounds of another pathname
    // So that we can trim from the right now.
    char * rString = pathname + strlen(pathname);
    while (rString > pathname && isspace((unsigned char)rString[-1]))
    rString--;
    *rString = '\0';
    return pathname;
}
```

I kept trying to make the function work throughout many renditions to see if I was in fact trimming incorrectly. It wasn’t until I had given up all hope and ctrl - z’d enough to iterate through and see if I ever did get it correct before I stopped and paused to see if other

debug statements I had put to check pathname mutation were doing the same thing. It was there that I finally noticed that I HAD PUT THE SPACE I WAS TRYING TO TRIM. Whether I was trimming correctly or not I never needed to.

fs_setcwd()

2. Pathname mutation on nested parsePath() calls.

Now that I had gotten back to square one realizing I did not ever need to trim the pathname I still was getting a mutated path.

```
+ //linux chdir
+ int fs_setcwd(char *pathname){
+     if (!pathname || strlen(pathname) == 0) return 0;
+     parsePathInfo ppi;
+     if (parsePath(pathname, &ppi) != 0);
+     printf("Error in fs_setcwd: Invalid path\n");
+     if (fs_isFile(ppi.lastElementName) == 0)
+     alreadyLoadedCWD = ppi.parent;
+ }
```

```
// Returns 1 if file, 0 otherwise;
int fs_isFile(char * filename){
    if (!filename || strlen(filename) == 0) return 0;
    parsePathInfo ppi;
    if (parsePath(filename, &ppi) != 0);
    printf("Error in fs_isFile: Invalid path\n");
    int index = findInDir(ppi.parent, filename);
    if (ppi.parent[index].isDir != 1) return 1;
    return 0;
}
```

Again at this point I had nested a fs_IsFile call to make sure I was changing CWD to something that was a dir. In the debug statement outputs each call to parsePath. I was getting obviously incorrect behavior like this.

In fs_setcwd() path name is: dir
In parsePath() path name is: .
In parsePath() path name is: .
In parsePath() pathname is invalid.

This led me to believe that because we were passing a non const char* it was getting changed. So I changed all char * pathname’s to const char *’s and got the same result. Then I tried to make a copy of the path passed in and only ever parsed the copy so the caller's pathname never gets mutated. That helped because I knew it was safer and

couldn't hurt. However the problem was the nested call to isFile that was calling parsePath in it shouldn't.
I resolved it by refactoring isFile to the final rendition you see without a parsePath call and simply made parsePath() do the copying only.

fs_setcwd()

3. Lack of robustness in handling changes.

```
// Case : Path refers to parent (eg. '/')
if (!ppi.lastElementName){
    //path already points to cwd
    alreadyLoadedCWD = ppi.parent;
    return 0;
}
// Assign local scope index from ppi struct for safety.
int index = ppi.index;
if (index < 0){
    printf("In fs_setcwd: could not resolve lastElementName.\n");
    return -1;
}
// Make sure we caller isn't trying to change to non-directory.
if (ppi.parent[index].isDir != 1){
    printf("fs_setcwd: %s is not a directory.\n", ppi.lastElementName);
    return -1;
}
// Case : Final element is dir and exists.
child = &ppi.parent[index];
// Load directory metadata off disk.
DE *newCWD = loadDirectory(child);
//printf("DEBUG cd: found '%s' at idx=%d, isDir=%d, startLBA=%d size=%d\n",
   //ppi.lastElementName, index, child->isDir,
   //child->blockNumber, child->numBytes);
if (!newCWD){
    printf("fs_setcwd: failed to load directory '%s'\n", ppi.parent[index].name);
    return -1;
}
// Free parent only if parsePath() loaded it dynamically from global cache's.
if (ppi.parent != alreadyLoadedRootDir && ppi.parent != alreadyLoadedCWD){
    free(ppi.parent);
}
// Assign global cache to correct directory.
alreadyLoadedCWD = newCWD;
```

Now that I had fixed all the stuff I had gotten wrong; errantly assuming it was the other function's fault I could better determine the flaws innate to my function. Between that and getting a better grasp of how to use the ppi struct I was able to improve the handling of checks necessary for the function.

I resolved it by acknowledging that errors could happen beyond the instance of trying to change to a file. I needed to handle if we gave it a path to root. I needed to load the Directory of where path points into a pointer. Finally I needed to check for mutations to

cache from parsePath() and finally set cache correctly to the pointer.

fs_setcwd()

```
char * fs_getcwd(char *pathname, size_t size){
+    /*// /Home/Documents/HW/csc..filesystem/
+    if (!pathname || size == 0)
+        return NULL;
+    if (!alreadyLoadedCWD || !alreadyLoadedRootDir)
+        return NULL;
+    if (alreadyLoadedCWD[0].blockNumber == alreadyLoadedRootDir[0]){
+        if (size < 2) return NULL;
+        pathname[0] = '/';
+        pathname[1] = '/0';
+        return pathname;
+    }
+    // csc-filesystem...
+    // Home/..
+    // ^ length of this
+    // add to start buffer becasue just a pointer
+    // move lenght into that
+    // /Documents
+    // ./../foo/bar/bla/..
+    // Home/bar
+    char *buf = malloc(size);
+    memset(buf, 0, size);
+    DE * currentDir = alreadyLoadedCWD;
+    strcat(buf, alreadyLoadedCWD->name);
+    // csc-filesystem
+    strcat(buf, "/");
+    // csc-filesystem/
+    while (currentDir[1].name != alreadyLoadedRootDir->name){
+        strcat(buf, currentDir[1].name);
+        // csc-filesystem/HW
+        strcat(buf, "/");
+        // csc-filesystem/HW/
+        currentDir = currentDir[1];
+        strcat(buf, currentDir[1].name);
+        // csc-filesystem/HW/Documents
+        strcat(buf, "/");
+        // csc-filesystem/HW/Documents/
+    }
+    char ** arrayOfSubStrings = malloc(size * 8);
+    int index = 0;
+    if (arrayOfSubStrings[i] != "." || arrayOfSubStrings[i] != ".." ) index++;
+    char * token = strtok(buf, "/");
+    // "/" "csc-filesystem" "/" .....
+    while (!token){
+        arrayOfSubStrings[index] = token;
+    }
```

This is the first draft that I could find within the repo. Let’s use this as a reference to better understand the flaws in my approach and the subsequent helpers I needed to implement to get to the final draft.

fs_setcwd()

What I thought would end up being the easiest components of the project let alone MS 2 very quickly revealed itself to be the nightmare it was. Let’s pick apart issues one at a time.

1. parsePath() was deceptively helpful.

Because I was operating off the very real fact that we needed parsePath to actually utilize a string to navigate our FS I initially assumed all the work had been done for me. I just needed to go along the path building the string as we go. Oh how I was wrong. The way we had set it up initially was correct, it gives us info of where the path points and nothing else.

I had incorrectly thought at first I could add some logic to it to build string as we go. Which not only turned out more complicated then I thought but I did not want to go and mess up Ian’s work to complete my function. I had to just accept that I needed to use the ppi struct as is but the problem was that it pointed to the end of path only.

2. Walking back up from CWD

After resigning myself to accepting that I only got the last element’s info I thought I could walk back up from there using the “..” entries as one can using cd making sure to add / in between the actual entries name. Not only was this becoming very convoluted but I realized very quickly I was just adding “..” the name could be used to load the directory if I did it correctly. But I wasn’t ever loading directories and using its entries name. I was simply walking back up until the names match root thinking that would be the end. All the while adding just “..”

```
int findChildByHeadBlock(const DE *parent, int entryCount, int childHeadBlock) {
    for (int i = 2; i < entryCount; i++) {                // skip '.' and '..'
        if (parent[i].name[0] == '\0') continue;          // empty slot
        if (parent[i].isDir && parent[i].blockNumber == childHeadBlock) {
            return i;
        }
    }
    return -1;
}
```

Eventually I did realize that I was not navigating the meta data incorrectly and I had to use block numbers to loadDirectory’s to even get access to the entry name. Furthermore the complexity of adding names was way more complex then first seemed. Now that I could add the names together to string even when doing so dynamically with a char ** I had the string in REVERSE ORDER...

3. How do I reverse the string AND add / delimiter in between?

This is the turning point where I stopped trying to hammer in a solution and accepted that pausing to rethink the problem, and doing a bit of extra work is more often than not the right answer. Although these issues are really two distinct problems the solution was embedded in answering both with one solution.

I was avoiding desperately needing to step back from the problem and doing more work to create a bonafide vector with the ability to dynamically grow and more easily separate elements of modular length

I.e. given path without delims reverse it and a
diruhohthisdirislongerdi/
How would I know how to separate?
And which do I do first, the reversal or adding '/'s ?

Finally I gave in and made an appropriately called stringbuilder because I realized that it would help me solve those problems much easier.

```
typedef struct stringBuilder{
    char **subStrings;
    // Length of vector
    int size;
    // How many elements vector can hold
    int capacity;
} stringBuilder;

//Vector functions
void initVector(stringBuilder* v);
int vectorAdd(stringBuilder* v, const char * substring);
char * build(const stringBuilder *v, char separator);
void freeVector(stringBuilder *v);
```

Note: This was not the beginning nor was it the end of fundamental programming lessons I had to finally learn via this project. But this specific example truly hammered it in for me. I knew that a vector was the answer relatively quickly, but I was being lazy and avoiding doing it because I thought “If I'm only gonna use it this once in this function (pwd) I will just try and do it in the function.” All the while actually making my job HARDER by doing less work.
This very clearly showed me my ignorance and I took this lesson to heart and adopted it into future issues.

4. Vector Implementation

Creation of stringBuilder struct, initVector(), vectorAdd(), freeVector() were relatively straightforward.

(Thankfully I had a vector to reference from Assignment 4 thanks Professor!)

However, I quickly ran into the issue of:

“Okay I have a working vector, Now what?”

While building the necessary components I realized that the reversal itself was relatively simple and I could do that before or after assuming I added ‘/’s correctly. Which led me to adding the build() function to correctly add delimiters.

```c
char * build(const stringBuilder *v, char separator){
    // Null check
    if (!v) return NULL;

    // Empty vector
    if (v->size == 0){
        char *emptyVector = (char *)malloc(1);
        if (!emptyVector) return NULL;
        emptyVector[0]='\0';
        return emptyVector;
    }

    // Get total length = sum of subStrings length + separators + '\0'
    size_t length = 1; //Start at one for atleast '\0'

    // Iterate along vector of substrings adding up their length in bytes.
    for (size_t i = 0; i < v->size; i++) {
        length += strlen(v->subStrings[i]);
    }

    // If we had "home" "joe" "documents"
    // size of the vector would be 3.
    // BUT we need to add separators
    // eg. "home" "/" "joe" "/" "documents
    // so we need two more spaces. i.e. o.g. size - 1
    length += (v->size - 1);

    // Create an out buffer returnString
    // to return with appropriate length
    char * returnString = (char *)malloc(length);
    // Check for malloc error
    if (!returnString) return NULL;
    // Create a string which we will use to iterate along vector's substrings
    // and add sepator: '/'
    // [string]
    // home joe documents
    char * position = returnString;
    for (size_t i = 0; i < v->size; i++){
        // So long as we aren't at the very first substring we add '/' first.
        if (i > 0) *position++ = separator;
        //Get length next substring to copy from.
        size_t ssLength = strlen(v->subStrings[i]);
        // Copy from substing to position string.
        memcpy(position, v->subStrings[i], ssLength);
        // Move the length of last substring forward.
        position += ssLength;
    }

    // Put null at end of position string.
    *position = '\0';

    return returnString;
}
```

5. build() complications

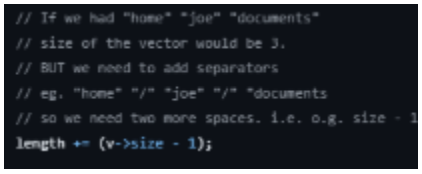
The final product is what is reflected above but you should pretty easily see the issue I ran into when navigating vectors to add /’s . I though ok well because we have char ** subStrings we can just use subStrings[i] to add each individual substring and then just add a / in between. I had the size field to show me how long the path name is so this should be easy right?

How do I determine how many /’s to add?

If I just used length of summation of substrings’ lengths and include null terminator. That doesn’t tell me how to know when to stop when I just add /’s in between using Substrings[i]. Because the length is growing as /’s are added so we’d end up finishing before we are truly done.

The breakthrough was in this one line of code:

length += (v->size - 1);



Here you can see the breakdown in my comment. But we COULD deterministically account for new lengths with appropriate arithmetic.

The next final fix was realizing I could use a char * position that I use to *move along* the path memcpying with added / as I go.

6. Okay we have the vector, way to add substrings, and add /’s now what?

Now all I had to do was integrate the vector into the get_cwd() which was relatively simple with conceptual improvements I made previously. It fundamentally boiled down to init’ing the vector, adding to it, and building in the correct places.

But that brings us to the still lingering issue that we have the pathname with / delimiters but in reverse order.

Although this was easier with the vector, it was more intricate than I thought. BUT believe it or not all that trimString NONSENSE ending up helping in the end. I vastly improved my ability to navigate, interpret, manage individual elements, change, and build with the

primitive char * data type.

```
// Now that we have added to the vector starting from cwd working up to root.
// The vector's substrings are in reverse order. So now we loop and reverse.
// Them to get them in our desired order.
if (parts.size > 1) {
   size_t endIndex = parts.size - 1;
   size_t startIndex = 0;
   while (startIndex < endIndex){
    char *tmp = parts.subStrings[startIndex];
    parts.subStrings[startIndex] = parts.subStrings[endIndex];
    parts.subStrings[endIndex] = tmp;
    startIndex++;
    endIndex--;
   }
}
```

Consequently, although it took some iterations this was all that needed to be done to do reversal but all the previous learned lessons are what made it easy.

Issues & Resolutions MS 3

Contiguous versus Discontiguous

Based on setbacks that occurred during MS2, and after meeting on 11/14 and discussing our options going forward for MS3, we have opted to continue with our contiguous implementation. We looked towards the end of the project and realized that keeping our contiguous implementation as-is would not cause headaches down the road, with the added benefit of not having to refactor the implementation we already had which resulted in ls, ls -la, md, pwd and cd working flawlessly. We figured that the main advantage of using discontiguous will come into play when there is a need to grow files. However, we realized that the only way files are going to be larger than 1 block in this project is if they are made in linux, then copied over using cp2fs. Therefore, no growing will be necessary since we have no way to grow them in our file system! (with text editor nano, etc.) In addition, in our project we will be making new files with touch, but they will always be a single block because they are new files! Thus the benefit of refactoring everything into discontiguous is negated. In any case, the lack of integration of key functionality of discontiguous structure in the previous milestone was the main deciding factor here.

parsePath()

Because parsePath was being called by essentially everything in the mfs layer, it became apparent after milestone 2 that it was pretty cumbersome and unsafe for the responsibility of making a working copy string of the pathname to be on the function caller, rather than handled internally so that the caller does not have to worry about it. We decided it was best to refactor this now before moving forward, even if it involved editing all the pre-existing calls to parsePath. In addition, the comment made by the professor (which we saved in our notes) suddenly makes sense, the while forever loop needs to strdup the token1 variable when the copy is handled internally! The refactor worked cleanly with the implementation and did not break any functionality.

Touch()

When we were going to start making headway on touch command we went to the shell to look and see what was going on when touch was called. We saw that all the work was actually being done by b_open(). That was the issue.

```
int cmd_touch (int argcnt, char *argvec[])
        {
#if (CMDTOUCH_ON == 1)
        int testfs_src_fd;
        char * src;

        switch (argcnt)
                {
                case 2: //only one name provided
                        src = argvec[1];
                        break;

                default:
                        printf("Usage: touch srcfile\n");
                        return (-1);
                }


        testfs_src_fd = b_open (src, O_WRONLY | O_CREAT);
        if (testfs_src_fd < 0)
        return (testfs_src_fd); //return with error

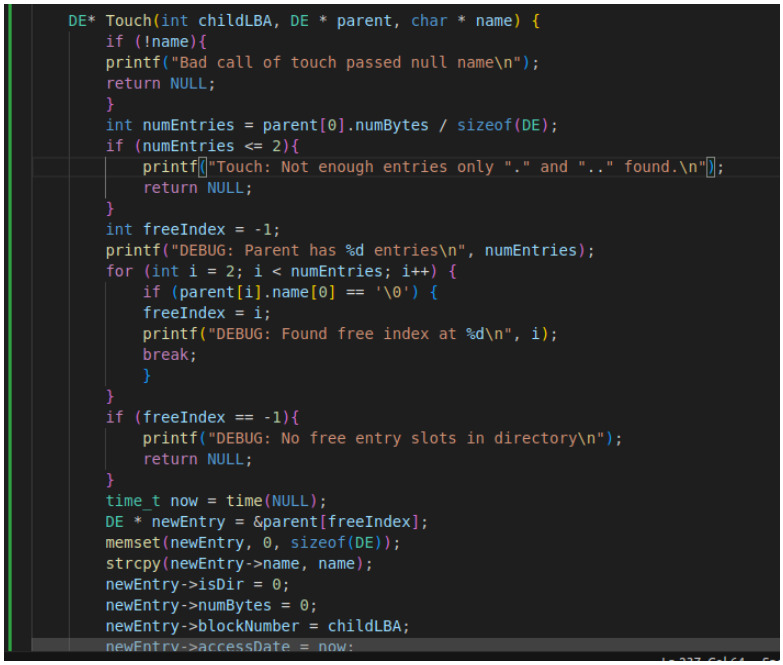
        b_close (testfs_src_fd);
#endif
        return 0;
        }
```

It was Thursday 11/13/25 , and during our conceptual brainstorming of functionality for MS3 we had initially thought about Ian and Joe handling all shell commands knowing that they couldn’t *really* be tested until Enrique had finished file functions. He had been going through some personal things and said he couldn’t work on anything until our Friday meeting on 11/14/25. We resolved it by realizing “Hey, we can just make a touch() helper function he can call once he creates the logic in b_open that checks for its needs”. And then we won’t have to wait for him to work on b_open. A win-win!

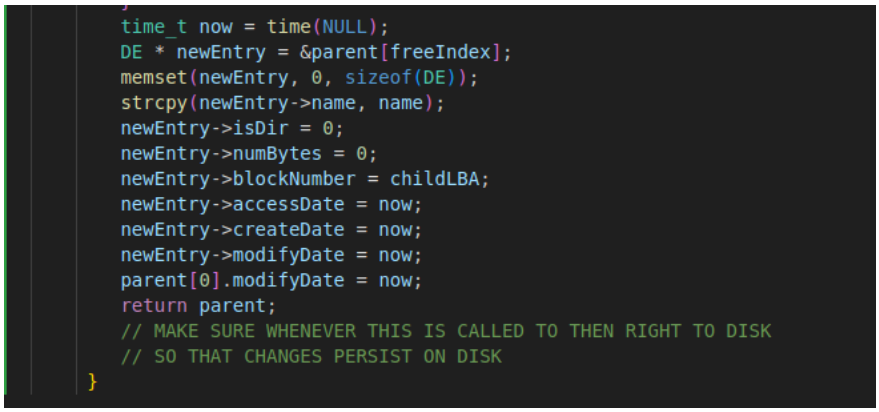
The next issue we faced was the extremely similar nature to createDir() and the need for similar logic used in mkDir(). Because of that we found ourselves often doing too much work because we thought it couldn’t be that simple.

Touch() continued…

We resolved it by stopping work and taking a break. During the time off and with the help of fresh eyes we narrowed down the exact parameters needed, deleted superfluous mallocs, and got rid of incorrect assignment of DE fields.



```
DE* Touch(int childLBA, DE * parent, char * name) {
    if (!name){
    printf("Bad call of touch passed null name\n");
    return NULL;
    }
    int numEntries = parent[0].numBytes / sizeof(DE);
    if (numEntries <= 2){
        printf("Touch: Not enough entries only "." and ".." found.\n");
        return NULL;
    }
    int freeIndex = -1;
    printf("DEBUG: Parent has %d entries\n", numEntries);
    for (int i = 2; i < numEntries; i++) {
        if (parent[i].name[0] == '\0') {
        freeIndex = i;
        printf("DEBUG: Found free index at %d\n", i);
        break;
        }
    }
    if (freeIndex == -1){
        printf("DEBUG: No free entry slots in directory\n");
        return NULL;
    }
    time_t now = time(NULL);
    DE * newEntry = &parent[freeIndex];
    memset(newEntry, 0, sizeof(DE));
    strcpy(newEntry->name, name);
    newEntry->isDir = 0;
    newEntry->numBytes = 0;
    newEntry->blockNumber = childLBA;
    newEntry->accessDate = now;
```



```
    time_t now = time(NULL);
    DE * newEntry = &parent[freeIndex];
    memset(newEntry, 0, sizeof(DE));
    strcpy(newEntry->name, name);
    newEntry->isDir = 0;
    newEntry->numBytes = 0;
    newEntry->blockNumber = childLBA;
    newEntry->accessDate = now;
    newEntry->createDate = now;
    newEntry->modifyDate = now;
    parent[0].modifyDate = now;
    return parent;
    // MAKE SURE WHENEVER THIS IS CALLED TO THEN RIGHT TO DISK
    // SO THAT CHANGES PERSIST ON DISK
}
```

As you can probably see, the touch function essentially is a bare-bones version of createDir that uses the logic of finding the free index we previously used in mkDir().

We then soon realized that touch will also need to update the fsm! We added a call to allocate() in the helper function because the fsm needs to

reflect the new block being taken up (even if there is nothing being written to it, because the new file is empty).

Touch continued…

During the next team meeting on 11/14, we discussed the plan moving forward with file related functions. There was some conceptual confusion in the group regarding the chain of what is being called by what to make touch happen. In the end after discussing, we all came to a better understanding that the mfs layer and the b_io layer are to mostly be treated as entirely separate and operate in different ways.

Based on the above realization, we came to the conclusion that the touch helper function we wrote in the mfs.c file was in the wrong place, even if the logic was correct. We had it returning a DE * type, but in reality we are going to need it to return a fileInfo type (like in assignment 5). We came up with the following plan of action: move the function prototype into fileinfo.h and retain the touch helper function from before, but switch the return type. This way, the b_io layer can still gather the necessary information from the mfs layer to populate the fileInfo, but the entire mfs layer won’t be exposed to the b_io layer!

b_write()

Here is the first draft of my b_write function

```
int b_write (b_io_fd fd, char * buffer, int count)
    if ((fd < 0) || (fd >= MAXFCBS))
        }
    // Step 2: Get pointer to the file's FCB entry;
    b_fcb *fcb = &fcbArray[fd];
    // Step 3: Check for Append flag.
    // Move the buffer forward to end of file.
    fcb->bufferStartPosition = fcb->fi->fileSize;
    // Step 4: Write Loop
    // Write until we reach number (count) passed in.
    size_t bytesWritten = 0;
    while (bytesWritten < count){
        // Step 5: Check to see if current file position
        // is in the current buffer window.
        // Our file pointer is behind the start of block             // Is our file position ahead of buffer
        // file starts at 700 ; buf[0] = byte 1024; WRONG FLUSH      // buf[0] = byte 0; file starts at 513; WRONG FLUSH
        // In both this cases buffer does NOT contain byte we want to write so we mush flush.
        if(fcb->currentFilePosition < fcb->bufferStartPosition || fcb->currentFilePosition >= fcb->bufferStartPosition + BLOCK_SIZE){
            // Flush buffer
            free(fcb->buf);
            // Find correct position
            int correctBufPos = (fcb->currentFilePosition - 511 / BLOCK_SIZE);
            fcb->bufferStartPosition = correctBufPos;
            int fileSizeInBlocks = (fcb->fi->fileSize - 511 / BLOCK_SIZE);
            LBAread(fcb->buf, fileSizeInBlocks, fcb->fi->location);
        }
        // Step 6: Compute current offset.
        size_t offset = fcb->currentFilePosition - fcb->bufferStartPosition;
        // Step 7: Determine number of bytes we can copy into buffer before end of block.
        int spaceInBlock = spaceInBlock - offset;
        // Step 8 : Update values
        // Take the minimum of space in block if only one block is left.
        // else use total amount of bytes left to copy.
        int bytesToCopy = spaceInBlock < count - bytesWritten ? spaceInBlock : count - bytesWritten;
        fcb->currentFilePosition += bytesToCopy;
        bytesWritten += bytesToCopy;
        // Take the max of current valid bytes or offset + bytestocopy
        fcb->numValidBufferBytes = fcb->numValidBufferBytes > offset + bytesToCopy ? fcb->numValidBufferBytes : offset + bytesToCopy;
        // Step 9: Check for full buffer.
        if (fcb->bufferStartPosition>=BLOCK_SIZE){
            LBAwrite(fcb->buf, fcb->buf + fcb->numValidBufferBytes; bytesToCopy);
            fcb->numValidBufferBytes=0;
        }
        // Step 10: If writing past old file size
        // Update fileInfo correctly
        if (fcb->currentFilePosition > fcb->fi->fileSize){
            fcb->fi->fileSize = fcb->currentFilePosition;
        }
    }
    return bytesWritten;
}
```

Here you will see a deluge of issues.

1. I forgot to put the if statement for append flag.
2. I was freeing the fcb->buffer to flush it instead of using LBAwrite()
3. I was computing correctBufPos incorrectly , should’ve been (currentPos / BS) * BS
4. Space in block was computed wrong it should be BLOCKSIZE - offset
5. I was checking for full buffer incorrectly should’ve used numValidBufBytes or offset + bytesToCopy and when either is equal to 512 its full)
6. I never had a memcpy to actually cpy from user buffer to i/o layer buffer

Here is everything else wrong and how I fixed it between the first and final draft.

7. The nesting of logic to check for if the buffer was outside the window got very messy and was not only hard to read but hard to implement.

I simplified it by just using a int as a proxy for a bool which would become valid if the condition was found;

```
int outsideWindow = (fcb->currentFilePosition < fcb->bufferStartPosition ||
fcb->currentFilePosition >= fcb->bufferStartPosition + BLOCK_SIZE);
```

8. I kept using wrong fi->location wrong because I wanted to do arithmetic but it was mutating the number because it points to that value.

```
// Flush buffer
int blockIndex = fcb->bufferStartPosition / BLOCK_SIZE;
int previousLocation = fcb->fi->location;
blockIndex +=  previousLocation;
LBAwrite(fcb->buf, 1, blockIndex);
fcb->numValidBufferBytes = 0;
```

I fixed it by just using an int to get the value then using previousLocation to correctly update proper index for LBAwrite()

9. Again because my checks for full buffers were very complicated it was hard for me to correctly figure out the correct logic

```
if(outsideWindow && fcb->numValidBufferBytes > 0){
    // Flush buffer
    int blockIndex = fcb->bufferStartPosition / BLOCK_SIZE;
    int previousLocation = fcb->fi->location;
    blockIndex +=  previousLocation;
    LBAwrite(fcb->buf, 1, blockIndex);
    fcb->numValidBufferBytes = 0;
}

if (outsideWindow && fcb->numValidBufferBytes == 0){
    // Realign the buffer
    fcb->bufferStartPosition = (fcb->currentFilePosition / BLOCK_SIZE) * BLOCK_SIZE;

    if (fcb->bufferStartPosition < fcb->fi->fileSize){
        // There is existing data in this block so we need to read it.
        int blockIndex = fcb->bufferStartPosition / BLOCK_SIZE;
        LBAread(fcb->buf, 1, fcb->fi->location + blockIndex);
        int remainingFileBytes = fcb->fi->fileSize - fcb->bufferStartPosition;
        fcb->numValidBufferBytes = (remainingFileBytes < BLOCK_SIZE) ? remainingFileBytes : BLOCK_SIZE;
    } else {
        // We're righting past EOF so we need to start with a 0'd block
        memset(fcb->buf, 0, BLOCK_SIZE);
        fcb->numValidBufferBytes = 0;
    }
}
```

Again using the bool proxy was useful, but fundamentally I was understanding bounds incorrectly. It just took time to think through

what I what data and where it was more to fix things. But again the formatting made a big difference.

10. Again I had a really rough time parsing out the proper logical flow. But once I had buffer flushes separated well and took care of the way I was keeping track of byteWritten, amount left in block, etc. were messy and wrong.

```
// Step 6: Compute current offset.
size_t offset = fcb->currentFilePosition - fcb->bufferStartPosition;
int spaceInBlock = BLOCK_SIZE - offset;

// Step 7: Determine number of bytes we can copy into buffer before end of block.
int remaining = count - bytesWritten;
int bytesToCopy = (remaining < spaceInBlock) ? remaining : spaceInBlock;

// Step 8 : Update values
// Take the minimum of space in block if only one block is left.
// else use total amount of bytes left to copy
memcpy(fcb->buf + offset, buffer + bytesWritten, bytesToCopy);
fcb->currentFilePosition += bytesToCopy;
bytesWritten += bytesToCopy;
// Take the max of current valid bytes or offset + bytestocopy
fcb->numValidBufferBytes = fcb->numValidBufferBytes > offset + bytesToCopy ? fcb->numValidBufferBytes : offset + bytesToCopy;
// Step 9: Check for full buffer. If full flush
int blockIndex = fcb->bufferStartPosition / BLOCK_SIZE;
if (fcb->numValidBufferBytes == BLOCK_SIZE || offset + bytesToCopy == BLOCK_SIZE){
    int previousLocation = fcb->fi->location;
    blockIndex += previousLocation;
    LBAwrite(fcb->buf, 1, blockIndex);
    fcb->numValidBufferBytes=0;
}
// Step 10: If writing past old file size
// Update fileInfo correctly
if (fcb->currentFilePosition > fcb->fi->fileSize){
    fcb->fi->fileSize = fcb->currentFilePosition;
}
```

Similar to previous messiness involved with if statements I took some time to learn how to use ternary operators. That combined with cleaning up. Helped relieve some of the cognitive bandwidth needed to piece out what exactly I needed to do and how to do it.

To summarize, I basically did nearly EVERYTHING wrong in my first draft. But I very quickly learned why programmers talking about if statements being bad and between that and giving in and learning how to use ternary operators proficiently. I finally was able to get to something that compiled. Now I just need to test it.

Chicken and the Egg problem with fsm and bytesRead

Soon after our first successful tests involving writing and reading multi-block files, we were testing out multiple reads and writes of large files and where they are being put into disk. Ian quickly noticed that the fsm was not being updated, which led to the discovery that the carry-over implementation of allocate() from when we were attempting to use discontiguos files will now be problematic.

The early stages of discovering the problem involved thinking about *where* writes were occurring in a contiguous fashion. The old allocate would simply find the lowest free block, then say “here you go caller, go ahead and

write here!”. This would have been fine in the event that our discontiguous implementation worked, but after our pivot to contiguous (see above) this became useless! What if there is a “sandwich” where the free block is in between two files? *Then allocate would need to know how large the file being written is before choosing the contiguous blocks!* However, this creates the Chicken and the egg problem…

Problem: If we are attempting to “sandwich” a new file into a gap that exists in between two previously written files, then allocate needs to be able to determine *whether or not the gap is sufficiently large to fit the file*. However, we do not know the size of the new file *until it has already been written!* This is due to the way our buffered writing works, i.e. the bytesWritten counter is accumulating while the write loop runs and is only an accurate representation of the size of the file *at the end of write!* This means by the time we are able to know how large the file is, we already have written it somewhere (almost certainly in the wrong place / trampling over pre-existing files).

Joe and Ian both analyzed this issue on their own time and came up with their own ideas to address it. Joe’s idea was essentially to mathematically predict the final value of bytesWritten using the information in the io layer, so that it could be passed to allocate before write. Ian’s idea was essentially to do a “dry-run” of write without actually committing to disk, which would allow early-access of the resulting bytesWritten, then to do a “real-run” once allocate has made it’s decision with bytesWritten. This sparked a lot of discussion and we ended up attending Professor Bierman’s office hours to present our problem and our individual ideas to address them. Professor Bierman told us that we were both essentially proposing the exact same solution, which was a relief. He gave us further ideas to address the issue.

What we ended up doing was rethinking the dependency flow. What if we just give write() a location that is guaranteed to not be in a sandwich? Then the issue is negated entirely because allocate can *retroactively* update the free space map once write is done!

The solution involved keeping track of a “freeTail” in the VCB. This is the block number following the highest used block number in the volume. I.e. the first free block directly after our last write. This “tail” is *guaranteed* to be a safe place to write, regardless of what the file size will end up being! The usage of a tail (basically a pointer to keep track of last written file) lets us avoid external fragmentation because we are always going to allocate in the range of all the free blocks. However, external fragmentation still inevitably pops up whenever we remove a file that does not meet our optimization condition (see section below). Not only does it help avoid inevitable external fragmentation it drastically helps us solve the chicken and egg issue. Because we always know where we can go using free blocks via tail we don’t even need to do preallocation NOR prediction. We simply use our current functionality of updateFileSizeOnDisk() which allows the io layer to

update meta data between io runs to do the allocation post run. All we had to do was make sure we nested an ensureAllocation function to check bits from location of file in bitMap with new update size to allocate them on first write that necessitates more blocks and ONLY when it needs more blocks.

Then after the fact, the allocate can ask “okay, how many blocks did you write?” and update the free space map after the fact. This of course increases the amount of external fragmentation in our file system, but we were willing to accept that.

rm optimization for freeTail system

With this new freeTail implementation allowing our multi-block file write trampling problem to be solved, it did come at the cost of more external fragmentation. Ian came up with a slight optimization to address this.

If a file/dir is deleted with the new freeTail implementation, the free space map does not have a chance to reuse any of the newly available space. It will still correctly mark the space as free, but because the freeTail is where all writes begin, there is never an opportunity to reuse those blocks. However, in the event that the last block of the file being deleted is one less than the freeTail index, that means that the freeTail can be moved left however many blocks get deleted. This means that the newly deleted space CAN be reused for other files. Simply put: if the file/dir to be deleted is the last file written, those blocks can be overwritten and reused. If not though, there will be a permanent gap where that file used to be, increasing external fragmentation.

b_close()

When looking at the provided shell we could see that changes would most obviously be finalized in close. The only thing was how if a file is grown/shrunk do we come up with a way for the changes to be reflected in higher layers.

We resolved it by coming up with the helper functions updateFileSizeOnDisk() and ensureAllocation().

updateFileSizeOnDisk()

```
// Function for io layer to keep metaData in FSM, VCB, and MFS layer in sync and on disk.
int updateFileSizeOnDisk(char *pathname, int newSize, int lba){
    if (!pathname)
        return -1;
    parsePathInfo ppi;
    // Point to appropriate file
    int ppResult = parsePath(pathname, &ppi);
    if (ppResult != 0){
        printf("In updateFileSizeOnDisk: parsepath called with invalid path.\n");
        return -2;
    }
    if (ppi.index < 0){
        printf("In updateFileSizeOnDisk: DE not found for:%s\n", pathname);
        return -3;
    }
    // Update size and date;
    ppi.parent[ppi.index].numBytes = newSize;
    ppi.parent[ppi.index].modifyDate = time(NULL);
    struct VCB *vcb = getVCB();
    // Do arithmetic to see if we went beyond one block initial allocation of DE's
    int blockSize = vcb->blockSize;
    // Get number of blocks to verify in bitmap.
    int blocksToCheck = (newSize + blockSize - 1) / blockSize;
    // Get how many blocks are not currently allocated and update them.
    int blocksToAllocate = ensureAllocation(blocksToCheck, lba);
    // Update if multiblock file was create then FSM was updated.
    if (blocksToAllocate > 1)
        printf("In updateFileSize on disk: %d new blocks allocated for filename: %s\n"
        , blocksToAllocate, ppi.lastElementName);
    // Persist those changes on disk.
    LBAwrite(ppi.parent, 1, ppi.parent[0].blockNumber);
    return 0;
}
```

updateFileSizeOnDisk() this was our first working draft but it was missing quite a few things conceptually even here. Furthermore there are more meta aspects about it that aren’t correctly illustrated in its iterations.

1. Because files sizes are changes in IO layer using MFS layers meta data how do we have them appropriate sync up without exposing all components to io layer().

A huge hurdle we had to get over conceptually going into MS 3 was what is the IO layers purpose, to what extent should it interact with previous layers, to what extent it NEEDS to see other layers, what paths of connection must those layers have.

In our first meeting for MS 3 this was hashed out for hours. Joe was of the camp that since we already had methods to read/write/open from disk we shouldn't NEED IO layer to do that. But because the professor mandated the use of IO layer, Joe deduced that this separate layer of abstraction should not be exposed to other layers (i.e. include them) because that would make the io layer obsolete.

2. Given that we need the io layer and it should not include other layers components BUT it can affect those layers metaData, how do we have it reflect that in the layers above?

This issue first propped itself up when we were initially trying to create a fileInfo struct to essentially use our previous io layer functions from assignment 5. But from our progress there we learned we could have a separate barebones fileinfo.h that has function prototypes and struct info therein, but has its implementation in the MFS layer.

This is what gave us the idea to do the same thing and declare a function prototype there that can speak back up to the MFS layer to have to correctly update metadata.

3. Now that we are correctly updated DE’s how do we make sure it stays in sync with other components e.g. VCB and FSM

Again, the inclination to just include all the layers into the io layer came roaring. Because of the aforementioned difficulties (see chicken and egg section above) we had already faced many issues trying to get the io layer to even correctly reflect DE’s metaData. The idea of figuring that all out again for other layers was beyond daunting.

The sunken cost fallacy actually comes to our benefit. Although Joe did end up going through hours of testing to verify if that was absolutely necessary. His trials doing so lead him to a breakthrough.

We were for sure at the point that we were correctly talking back up to the MFS layer in updateFileSizeOnDisk() in close. The implementation was already in the MFS layer so it had access to other components. We just had to figure out how we could weave io interactions results correctly into functions that would reflect made changes to all components.

Throughout his testing and implementation of ensureAllocation() (see above) a function to make sure file growth is reflected in FSM and appropriate tail movement in VCB he realized he could just do it in updateFileSizeOnDisk() using tools from other layers specifically looping with checkBit() using the accessor fsmIsUsed().

4. File growth is handled in FSM. What about shrinkage?

We had previously assumed shrinkage was limited to fs_delete which indeed already handled FSM interactions. However, when going to correct handling of flags in io layer it was revealed that we weren’t handling one specific case.

What if we had a file that grew be given the TRUNCATE flag?

When it’s size is set to 0, what if it stays as such?

Because we already handled growth reflection in FSM via ensureAllocation(). All we had to do was the reverse. We already had the logic. So fundamental growth logic of ensure allocation was just put in updateFileSize on disk and reversed.

cp2fs and cp2l corrections

At this point we had all of our FS working. Everything was talking to each other correctly, obscured as needed, and all shell commands were giving correct output.

However, given all of the logic up to this point specifically regarding our implementation of the test being nested in getFileInfo we would create a file if it didn’t exist. Which was fine for everything. In fact, we assumed that because the shell was a dilution of complexity of BASH i.e. (mv not being given pathname but instead always given just a file name). We reasonably assume it probably wouldn't be an issue that when we cp2fs a file that did not exist in linux it would touch an empty file of that name.

Yet, in an effort to dot all our i’s and cross all our t’s Joe went to see if the assumption that we couldn't mutate any aspects of the shell at least explicitly stated was correct this whole time.

I have provided an initial “main” (fsShell.c) that will be the driver to test you file system. Your group can modifiy this driver as needed. The driver will be interactive (with all built in commands) to list directories, create directories, add and remove files, copy files, move files, and two “special commands” one to copy from the normal filesystem to your filesystem and the other from your filesystem to the normal filesystem.

You should modify this driver as needed for your filesystem, adding the display/setting of any additional meta data, and other functions you want to add.

It was here that I saw that we could and should make modifications as needed.

So albeit potentially unnecessary for scope of the project Joe went and made a fix to the shells logic of cp2fs and cp2l.

```
testfs_fd = b_open (src, O_RDONLY);
if (testfs_fd < 0){
    printf("filename:%s does not exit in our filesystem.\n", src);
    return -1;
}
```

```
linux_fd = open (src, O_RDONLY);
if (linux_fd < 0){
    close(linux_fd);
    printf("Filename:%s does not exist in on Linux Side.\n", src);
    return -1;
}
```

Although minor, these changes were all that were necessary. All we had to do was make sure that a file exists before we go to copy. That way we never touch a file unintentionally and furthermore this is how it should be. You should not be able to copy files that do not exist.

James contributions for MS 3 being empty.

We had been trying to help James throughout the majority of the project. Whether it meant multi-hour calls helping him fix his functions. Giving him less complicated stuff to do. Taking responsibilities he said he ended up not being able to handle. The list goes on, but for that reason we initially gave him cp2fs and cp2l to handle.
In the first meeting following the end of MS2 we discussed how we discovered that all of MS 3’s shell commands were done by IO layer. Ian, Enrique, and Joe decided to all cooperate on IO layer together over the weekend since James said he wouldn't be available Fri-Sunday.
Come Sunday we were basically done with the IO layer so in an effort to include him we gave him a chance to help us catch up on descriptions of all our functions. Come the next meeting he told us that he wouldn’t have them done for two weeks at least i.e. the full extension. Given that we were all done with our FS by this point and figured we all could have it done in 3 days at most the call was made to just handle our own functions descriptions in the writeup like originally.

Descriptions Of Functions MS 1

(Joe) Load FSM()

```c
int loadFSM (int blockSize, unsigned int startBlock){
    size_t numBytes = 5 * blockSize;
    if (fsm) {
        free(fsm);
        fsm = NULL;
        return -1;
    }
    fsm = (char*)malloc(numBytes);
    if (!fsm) return -1;
    uint64_t blocksRead = LBAread(fsm, 5, startBlock);
    if (blocksRead != 5){
        free(fsm);
        fsm = NULL;
        return -1;
    }
    return 0;
}
```

Load FSM() : Although we had the functionality to init and write to FSM, we did not have a way to load it from disk. Here we simply take in block size and LBA position of FSM to read it from disk. Note: we hard-coded 5 blocks because we did not store numblocks of FSM in VCB but instead just relied on our meta knowledge knowing that region.

(Joe & James) createDir()

```c
DE* createDir(int childLBA, int parentLBA, int minEntries, DE * parent, int blockSize) {
    //4a) How much memory neeeded
    //4b) Number of entries wanted is minEntries parameter.
    //4c) Multiply size of your directory entry by number of entry to get number of bytes needed.
    // Make sure we never have less then 2 directories.
    if (minEntries < 2) { minEntries = 2;}
    size_t bytesNeeded = minEntries * (int)sizeof(DE);
    //4d) How many blocks needed.
    size_t blocksNeeded = (bytesNeeded + (blockSize - 1)) / blockSize;
    size_t bytesToMalloc = blocksNeeded * blockSize;
    // How many entries could fit in that space
    size_t actualEntriesCount = bytesToMalloc / (int)sizeof(DE);
    //printf("In createDir(): Making %ld entries", actualEntriesCount);
    if (blocksNeeded != 1) {
        //Multiple Block Dirs not implemented yet.
        return NULL;
    }

    DE * dir = (DE *)malloc(bytesToMalloc);
    if (!dir) { return NULL;}
    //Make clean slate.
    memset(dir, 0, bytesToMalloc);

    //4e) Initiliaze entries
    // Initialize all entries as empty
    for (int i = 2; i < actualEntriesCount; i++) {
        dir[i].name[0] = '\0';
        dir[i].isDir = 0;
        dir[i].numBytes = 0;
        dir[i].blockNumber = -1;
        dir[i].loc = -1;
    }
    //int rootPhys = 6;
    time_t now = time(NULL);
    //4f) Allocate blocks (one block for root directory)
    if (childLBA < 0 || parentLBA < 0){
        free(dir);
        return NULL;
    }
```

(Joe & James) createDir()

```
// Setup "." entry (current directory)
strcpy(dir[0].name, ".");
dir[0].numBytes = actualEntriesCount * (int)sizeof(DE);
dir[0].isDir = 1;
dir[0].createDate = now;
dir[0].modifyDate = now;
dir[0].accessDate = now;
dir[0].blockNumber = childLBA;
//dir[0].loc = rootPhys;
//dir[0].blockNumber = rootPhys;

// Setup ".." entry (parent directory)
if (!parent) {
    // Root directory - parent points to itself
    parent = dir;
}

//4h)
strcpy(dir[1].name, "..");
dir[1].numBytes = parent->numBytes;
dir[1].isDir = 1;
dir[1].createDate = dir[0].createDate;
dir[1].modifyDate = dir[0].modifyDate;
dir[1].accessDate = dir[0].accessDate;
dir[1].blockNumber = parentLBA > 0 ? parentLBA : childLBA;
//dir[1].blockNumber = rootPhys;
//dir[1].loc = rootPhys;
//4i)
//writeDirToDisk(dir, blockSize);
//4j)
//printf("In createDir() : Root logical=%d physical=%d\n",
//v->blockIndexRootDirectory, v->blockIndexRootDirectory + 1);
return dir;
```

createDir() was our DE function to create a new directory. It takes in LBA locations for parent and child, pointer to parent where child is written as well as block size. It does arithmetic to figure out numEntries that fit in a block, initializes them to empty, then assigns the first two entries to “.” and “..” respectively for path traversal.

(Joe & James) writeDirToDisk()

```c
void writeDirToDisk(const DE * dir, int blockSize) {
    //This version of writing directory is only writing one block.
    //For next milestone we will implement linked functionality to make it discontiguous.
    //Remember to refactor this function LATER!!!!
    size_t blocksToWrite = (dir[0].numBytes + blockSize - 1) / blockSize; // Should just be one.
    if (blocksToWrite <= 0) blocksToWrite = 1;
    size_t blockIndex = dir[0].blockNumber;
    //printf("Dir block number: %d\n", dir[0].blockNumber);
    printf("writeDirToDisk: writing : %ld blocks to LBA: %d\n", blocksToWrite, dir->blockNumber);
    LBAwrite((char*)dir, blocksToWrite, dir[0].blockNumber);
    /*for (int i = 0; i < blocksToWrite; i++){
        //int blockIndex = getNthBlockOfChain(dir->loc,i);

        LBAwrite((char*)dir + i*blockSize, 1, blockIndex);
        //char * [numBytes...loc....blocknumber...]
    }*/
    fsmWrite();
}
```

writeDirToDisk() is a function that we call when we have made a change to a DE i.e. meta-data that we want to store on disk (sample volume) using provided LBAwrite() to do so. Making sure to update the FSM appropriately.

(Joe and Enrique) initFileSystem()

```c
int initFileSystem (uint64_t numberOfBlocks, uint64_t blockSize){

    printf ("Initializing File System with %ld blocks with a block size of %ld\n", numberOfBlocks, blockSize);
    /* TODO: Add any code you need to initialize your file system. */
    // Check if filesystem already exists by reading block 0 and checking magic number
    char* testBuffer = (char*)malloc((size_t)blockSize);
    if (testBuffer == NULL){
        printf("Error: Failed to allocate memory for filesystem check\n");
        return -1;
    }

    memset(testBuffer, 0, (size_t)blockSize);
    // Try to read VCB at block 1
    uint64_t blocksRead = LBAread(testBuffer, 1, 0);

    // Check signature to see if we need to format volume
    int filesystemExists = 0;
    if (blocksRead == 1){
        // Cast to VCB struct to check magic number
        struct VCB* testVCB = (struct VCB*)testBuffer;
        if (testVCB->MagicNo == VCB_MAGIC){ filesystemExists = 1;}
    }
    free(testBuffer);


    // File system found path.
    if (filesystemExists){
        // Existing filesystem - read VCB from disk
        printf("Existing filesystem detected. Loading VCB...\n");
        if (readVCB(blockSize) != 0){
            printf("Error: Failed reading VCB from existing filesystem.\n");
        }
        struct VCB* v = getVCB();
        if (loadFSM(blockSize, v->blockIndexFSM) != 0){
            printf("Error: Failed to load FSM from disk.\n");
            return -1;
        }
        DE rootStub = {0};
        rootStub.isDir = 1;
        rootStub.blockNumber = v->blockIndexRootDirectory;
        alreadyLoadedRootDir = loadDirectory(&rootStub);
        alreadyLoadedCWD = alreadyLoadedRootDir;
        return 0;
```

(Joe and Enrique) initFileSystem()

```
// File system found path.
if (filesystemExists){
    // Existing filesystem - read VCB from disk
    printf("Existing filesystem detected. Loading VCB...\n");
    if (readVCB(blockSize) != 0){
        printf("Error: Failed reading VCB from existing filesystem.\n");
    }
    struct VCB* v = getVCB();
    if (loadFSM(blockSize, v->blockIndexFSM) != 0){
        printf("Error: Failed to load FSM from disk.\n");
        return -1;
    }
    DE rootStub = {0};
    rootStub.isDir = 1;
    rootStub.blockNumber = v->blockIndexRootDirectory;
    alreadyLoadedRootDir = loadDirectory(&rootStub);
    alreadyLoadedCWD = alreadyLoadedRootDir;
    return 0;
    // Load root directory into memory so available for
    // path parsing operations. Ensures parsePath can
    // always find the root directory when processing absolute paths.
    //if (loadRootDirectory() != 0) {
        //printf("Error: Failed to load root directory\n");
        //return -1;
    //}
    } else {
        // Fresh filestem path
        // New filesystem - initialize and write VCB
        printf("New filesystem detected. Initializing VCB...\n");
        if (initVCB(numberOfBlocks, blockSize) != 0){
        printf("Error: Failed to initialize VCB\n");
        return -1;
        }
        // Initilize FSM (Mark 1..6 used; and write to blocks 2..6)
        if(initFSM(blockSize) != 1){
            printf("Error: Failed to initialize FSM\n");
            return -1;
        }
        struct VCB *v = getVCB();
        // Create Root Directory (should allocate to block 7)
        u_int64_t rootLBA = v->blockIndexRootDirectory;
        DE *rootDir = createDir(rootLBA, rootLBA, 2, NULL, v->blockSize);
        /*if (rootDir == NULL){
            printf("Error: Failed to create directories.\n");
            return -1;
        }*/
        // Cache the newly created root directory in memory.
        // createDir returns a pointer to the directory in memory,
        // so we can use it directly without reading from disk.
```

```
            rootDir[0].blockNumber = rootLBA;
            rootDir[0].loc = rootLBA;
            fsmReserveLBA(rootLBA);
            fsmWrite();
            rootDir[1].blockNumber = rootLBA;
            rootDir[1].loc = rootLBA;
            writeDirToDisk(rootDir, v->blockSize);
            alreadyLoadedCWD = rootDir;
            alreadyLoadedRootDir = rootDir;
            writeVCB();
            //alreadyLoadedRootDir = loadDirectory(&rootStub);
            //alreadyLoadedCWD = rootDir;  // Start at root
            return 0;
        }
}
```

initFileSystem() is our function that we have in fsInit.c that provides necessary logic to boot up or filesystem with two main branches for whether it is starting fresh (no volume) or if volume exists i.e. VCB found with matching signature. We first create a buffer of bytes the size of VCB and read from block it is expected to be at LBA 0. We use a proxy bool fileSystem exists and set it to 1 if VCB has a matching signature.

If there is a match we create a rootStub to correctly load the rootDirectory and then appropriately set our global caches tracking loaded root and CWD.

If no match is found we know we need to initialize the FS so we call our init functions. createDir with NULL (i.e. root) and set the appropriate location using indices stored in VCB. Making sure to cache root and set CWD to root.

initFSM() The initFSM function is what mallocs the global static fsm, sets all blocks to 0 (free), then marks block 1 as used (VCB) and blocks 2-6 as used (FSM), marks block 7 as used (Root directory), and writes the map to disk.

Allocate() The allocate function does not need to return a starting block number of a grouping of contiguous available blocks because our files are going to use discontiguous linked lists. This means allocate simply finds the next free block, marks it as used, and returns that block number for the caller to use.

setBit() Is a static function that directly manipulates our bitmap using bitwise logic operations. Using the inputted block index, it finds the corresponding byte in which that bit will be, then determines which bit within that byte is the desired one using modular arithmetic. It then creates a mask that will be used to bitwise OR the entire byte. The mask for setbit is seven zeros and a single 1, with the 1 being in the correct position of “bitInByte” from the modular arithmetic. For example, 00100000. When this mask is bitwise ORed with the byte in question, that particular bit is then set to 1 without changing any of the surrounding bits. Then this result is assigned to the byte in the bitmap to overwrite it.

getBit() Is a static function that queries the block index in question. Similar to setBit, it creates a mask which it will use to interact with the byte in which that bit resides. Similar to setbit, the mask will be made to be seven 0s and one 1 with the 1 positioned

in the spot where the queried bit is. When this mask is bitwise ANDed with the byte, the result will be a 0 or 1 which reflects the value of the bit in question. Rather than assigning this value to anything in the bitmap, it is simply returned to the caller. This operation does not alter the bitmap.

clearBit() Is a static function that sets the blockindex in question to 0. Unlike the setBit and getBit however, clearbit interacts with the bitmap using a different mask. It is the inverse of those, seven 1s and a single 0 where the desired bit is. The mask is calculated the same way, but with the addition of flipping the bits using the bitwise NOT operator : ~. This allows the bitwise AND with the byte in question to result in affecting only the 0 position which always fails the and condition, replacing a 0 with whatever was there before. The other bits are unaffected by this (i.e. 1 and 1 is 1, 1 and 0 is 0) and therefore the result is written to that byte spot in the fsm.

fsmWrite() Simply writes the in memory version of the fsm (the global fsm variable) to disk at exactly block 1 for 5 disks (this will never change so it is hard coded) using LBAwrite().

(Enrique) initVCB()

```c
// Initialize a new VCB and write it to disk (for formatting new filesystem)
int initVCB(uint64_t numberOfBlocks, uint64_t blockSize)
    {
    // Store block size for future operations
    vcbBlockSize = blockSize;

    // Allocate buffer for one block (VCB will be stored in this block)
    char* buffer = (char*)malloc(blockSize);
    if (buffer == NULL)
        {
        printf("Error: Failed to allocate memory for VCB\n");
        return -1;
        }

    // Zero out the buffer to ensure clean state
    memset(buffer, 0, blockSize);

    // Cast buffer to VCB struct pointer
    struct VCB* vcb = (struct VCB*)buffer;

    // Set VCB fields
    vcb->MagicNo = VCB_MAGIC;
    vcb->numBlocks = (unsigned int)numberOfBlocks;
    vcb->blockSize = (unsigned int)blockSize;
    vcb->numFreeBlocks = numberOfBlocks - 7;
    vcb->freeTail = 7;
    // Due to partition table being at physical block address 0, vcb is at lba 0, but really pba 1.
    // Subsequently fsm is at lba 1 pba 2.
    vcb->blockIndexFSM = 1;
```

```c
    // Root directory will start after VCB (lba 0) and FSM (lba's 1-6)
    // So root directory starts at lba 6 phys 7.
    vcb->blockIndexRootDirectory = 6;

    // Write VCB to disk at block 1
    uint64_t blocksWritten = LBAwrite(buffer, 1, 0);
    if (blocksWritten != 1)
        {
        printf("Error: Failed to write VCB to disk\n");
        free(buffer);
        return -1;
        }

    // Store VCB in memory (allocate separate memory for persistent storage)
    vcbInMemory = (struct VCB*)malloc(sizeof(struct VCB));
    if (vcbInMemory == NULL)
        {
        printf("Error: Failed to allocate memory for VCB storage\n");
        free(buffer);
        return -1;
        }

    // Copy VCB data to persistent memory
    memcpy(vcbInMemory, vcb, sizeof(struct VCB));

    // Free temporary buffer
    free(buffer);

    printf("VCB initialized: MagicNo=0x%X, numBlocks=%u, blockSize=%u, FSM=%u, RootDir=%u, NumFreeBlocks=%u\n",
        vcbInMemory->MagicNo, vcbInMemory->numBlocks, vcbInMemory->blockSize,
        vcbInMemory->blockIndexFSM, vcbInMemory->blockIndexRootDirectory, vcbInMemory->numFreeBlocks);

    return 0;
    }
```

initVCB() The initVCB() function is used when you're first creating or formatting a new filesystem. The Volume Control Block (VCB) is usually the first block of a volume and contains boot control information. It's like the “header” of your filesystem that stores all the important information about how the filesystem is organized. When you call this function, you pass in two parameters: the total number of blocks in your volume and the size of each block. The function then does several things. First, it allocates a buffer in memory that's big enough to hold one block—this is where we'll build our VCB structure before writing it to disk. The function uses malloc() to get this memory, and if that fails, it returns an error. Next, the function zeroes out the entire buffer using memset() to make sure we start with a clean slate, since we don't want any leftover data interfering with our VCB. Then it casts this buffer to a VCB struct pointer so it can set all the fields.The function sets up several important pieces of information in the VCB. One key field is the magic number (VCB_MAGIC), which identifies what type of filesystem this is. When the filesystem is read later, it looks at this magic number and says “hey, is this my drive?”—it's basically a signature that tells the system “yes, this is the right kind of filesystem.” It also stores the total number of blocks and the block size, and calculates how many free blocks are available. The function also sets where the Free Space Map starts and where the root directory begins. It’s important to understand that these aren’t actual pointers—you can’t use a pointer to point to something on disk because that’s not a memory location. Instead, these are block indices or block numbers that tell us where on the disk these structures are located. Later, when we need to access this information, we’ll read the VCB from disk into main memory, cast that memory as a pointer to a VCB structure, and then we can reference the information. Once all the VCB fields are set up, the function writes this block to disk at logical block address 0 (LBA 0) using the LBAwrite() function. This is crucial because this information needs to be written to disk—you can’t just start referencing it without first bringing it into main memory. If the write fails, the function cleans up the memory and returns an error. After successfully writing to disk, the function also keeps a copy of the VCB in memory. This is useful because other parts of the filesystem will need to access this information frequently, and reading from memory is much faster than

reading from disk every time. The function allocates separate memory for this persistent storage and copies the VCB data there using memcpy(). Finally, the function prints out a confirmation message showing all the key values that were set and returns 0 to indicate success. If anything goes wrong along the way (memory allocation fails, disk write fails, etc.), it prints an error message, cleans up any allocated memory, and returns -1.

(Enrique) readVCB()

```c
// Read VCB from disk (block 1) into memory
int readVCB(uint64_t blockSize)
    {
    // Store block size for future operations
    vcbBlockSize = blockSize;

    // Allocate buffer for one block
    char* buffer = (char*)malloc(blockSize);
    if (buffer == NULL)
        {
        printf("Error: Failed to allocate memory for VCB read\n");
        return -1;
        }

    // Zero out the buffer
    memset(buffer, 0, blockSize);

    // Read block 1 from disk
    uint64_t blocksRead = LBAread(buffer, 1, 0);
    if (blocksRead != 1)
        {
        printf("Error: Failed to read VCB from disk\n");
        free(buffer);
        return -1;
        }

    // Cast buffer to VCB struct pointer
    struct VCB* vcb = (struct VCB*)buffer;
```

```
// Verify magic number to ensure valid filesystem
if (vcb->MagicNo != VCB_MAGIC)
    {
    printf("Error: Invalid VCB magic number (0x%X). Expected 0x%X\n",
        vcb->MagicNo, VCB_MAGIC);
    free(buffer);
    return -1;
    }

// Allocate memory for persistent storage if not already allocated
if (vcbInMemory == NULL)
    {
    vcbInMemory = (struct VCB*)malloc(sizeof(struct VCB));
    if (vcbInMemory == NULL)
        {
        printf("Error: Failed to allocate memory for VCB storage\n");
        free(buffer);
        return -1;
        }
    }

// Copy VCB data to persistent memory
memcpy(vcbInMemory, vcb, sizeof(struct VCB));

// Free temporary buffer
free(buffer);

printf("VCB loaded: MagicNo=0x%X, numBlocks=%u, blockSize=%u, FSM=%u, RootDir=%u\n",
    vcbInMemory->MagicNo, vcbInMemory->numBlocks, vcbInMemory->blockSize,
    vcbInMemory->blockIndexFSM, vcbInMemory->blockIndexRootDirectory);

return 0;
}
```

readVCB() The readVCB() function is used when you're starting up your program and need to work with an existing filesystem that was previously created. This function reads an existing VCB from disk and brings it into main memory so the filesystem can use it. When you call this function, you pass in the block size as a parameter. The function needs to know the block size so it can allocate the right amount of memory for reading the VCB from disk. First, it stores this block size for future operations, then it allocates a buffer in memory that's big enough to hold one block. If the memory allocation fails, the function returns an error. Next, the function zeroes out the buffer to make sure we start clean, then it reads the first block from disk using the LBAread()function. This is where we actually bring the VCB from disk

into main memory. If the read fails or doesn't read exactly one block, the function prints an error, cleans up the memory, and returns an error.Once the data is read from disk into the buffer, the function casts this buffer to a VCB struct pointer so it can access the VCB fields. We read the raw bytes from disk, and now we're interpreting them as a VCB structure.Before trusting this data, the function verifies the magic number to make sure this is actually a valid filesystem. It checks if the magic number in the VCB matches the expected VCB_MAGIC value. This is important because if someone tries to read from a disk that doesn't have our filesystem on it we want to catch that early. If the magic number doesn't match, the function prints an error message showing what it found versus what it expected, cleans up the memory, and returns an error.If the magic number is valid, the function then allocates memory for persistent storage of the VCB if it hasn't already been allocated. This is where we'll keep a copy of the VCB in memory for quick access. The function copies the VCB data from the temporary buffer into this persistent memory location using memcpy(), then frees the temporary buffer since we no longer need it.Finally, the function prints out a confirmation message showing the key values that were loaded from disk, and returns 0 to indicate success. Now that the VCB is in memory, other parts of the filesystem can access this information quickly without having to read from disk every time.

(Enrique) writeVCB()

```c
// Write VCB from memory to disk (block 1)
int writeVCB(void)
    {
    if (vcbInMemory == NULL)
        {
        printf("Error: VCB not loaded in memory\n");
        return -1;
        }

    // Allocate buffer for one block
    char* buffer = (char*)malloc(vcbBlockSize);
    if (buffer == NULL)
        {
        printf("Error: Failed to allocate memory for VCB write\n");
        return -1;
        }

    // Zero out the buffer
    memset(buffer, 0, vcbBlockSize);

    // Copy VCB to buffer
    memcpy(buffer, vcbInMemory, sizeof(struct VCB));

    // Write to disk at block 1
    uint64_t blocksWritten = LBAwrite(buffer, 1, 0);
    if (blocksWritten != 1)
        {
        printf("Error: Failed to write VCB to disk\n");
        free(buffer);
        return -1;
        }

    // Free temporary buffer
    free(buffer);

    return 0;
    }
```

writeVCB() The writeVCB() function is used when you need to save changes to the VCB back to disk. This is important because while the VCB is kept in memory for quick access, any changes you make to it only exist in memory until you write them back to disk. If you don't

write the VCB back to disk, those changes will be lost when the program ends. When you call this function, it doesn't take any parameters because it uses the VCB that's already stored in memory. The function checks if there's actually a VCB loaded in memory—if vcbInMemory is NULL, that means the VCB was never read from disk, so there's nothing to write. In that case, it prints an error and returns an error code. If the VCB is in memory, the function allocates a temporary buffer that's big enough to hold one block. This buffer is where we'll prepare the VCB data before writing it to disk. If the memory allocation fails, the function returns an error.Next, the function zeroes out the buffer to make sure we start with a clean slate, then it copies the VCB data from the in-memory copy (vcbInMemory) into this temporary buffer using memcpy(). This prepares the data in the format needed for writing to disk. Once the buffer is ready, the function writes it to disk using the LBAwrite() function. This is where the changes actually get saved to disk. If the write fails or doesn't write exactly one block, the function prints an error, cleans up the temporary buffer, and returns an error. After successfully writing to disk, the function frees the temporary buffer since we no longer need it, and returns 0 to indicate success. Now the VCB on disk matches what's in memory, so the changes are persisted and will still be there the next time the filesystem is loaded.

(Enrique) getVCB()

```
// Get pointer to in-memory VCB (for other modules to access)
struct VCB* getVCB(void)
    {
    return vcbInMemory;
    }
```

getVCB() The getVCB() function is a simple helper function that gives other parts of the filesystem access to the VCB that's stored in memory. It doesn't take any parameters and just returns a pointer to the in-memory VCB structure. If the VCB hasn't been loaded into memory yet (either through initVCB() or readVCB()), this function will return NULL, which is a signal to the caller that the VCB isn't available.

Descriptions Of Functions MS 2

(Ian) parsePath() and findInDirectory

parsePath is a crucial piece of the mfs layer, as most other functions rely on it to gather useful information about the directory entry associated with that path, which is in string form.

It first determines if the path is absolute or not and can handle both cases. It then tokenizes the string on ‘/’ and walks through each piece one by one in the while forever loop (assuming the path is not just root “/”) For each token, it searches for the matching name within the parent directory. If that matches and is a directory, and there are more tokens in the path, it will then load that directory and repeat the process.

When it reaches the final token, (i.e. token2 is NULL), the index of token1 populates the ppi index and token1 populates the lastElementName, and the parent, which is now the directory that contains the final element, is populated in the ppi. The ppi instance is returned, and now the caller has usable info about the path that they inputted.

parsePath relies on the findInDirectory helper function, which searches through the DEs in a directory iteratively checking for a name match. If a match is found, that index is returned to the caller.

parsePath also handles several errors, in which case it returns a unique negative number so that the caller knows specifically what went wrong.

(Ian) fs_mkdir()

Mkdir takes a pathname and creates a new directory in that position that is empty except for a fresh . and .. entry. It makes use of parsePath to confirm that the path is valid and to utilize information about the path.

To do this, mkdir first allocates a space for the directory to go. For milestone two, this was done using the legacy version of allocate(), which would search for the first free block and assign it. However, see milestone 3 for the freeTail refactor. The final refactor means that mkDir always uses the freeTail as its location to write the dir to.

After the space is allocated, mkdir makes use of createDir which initializes the clean . and .. into the new spot.

The last step is connecting this new dir to the parent by finding a free DE slot in the parent, and populating it with a new DE (and corresponding fields) for the new dir.

The final step is making use of writeDirToDisk for both the new dir entry and the updated parent.

(Ian) fs_rmdir()

rmdir works as the counterpart to mkdir, given a pathname it will find and delete a dir. It also checks for various errors and returns unique negative numbers to indicate to the caller what specifically went wrong.

The main check before proceeding with the removal is to ensure that the directory is empty. This means that the only DEs in the dir are dot and dot dot. If any entries are detected past index 1 (dot dot), rmdir does not proceed and error message prints.

Once the directory is confirmed to be empty, the removal process begins. It is similar to the process in mkdir but doing the opposite. First rmdir clears the bit where the dir was in the free space map, updating the vcb trackers numFreeBlocks and freeTail. Second, the parent’s DE entry is cleared and the parent is written to disk to reflect changes.

(Joe) loadDirectory()

```c
// Joe
// Function for reading our DE metadata off disk w/ LBAread()
DE *loadDirectory(const DE *dirEntry){
    // Print what we were asked to load (helps you spot the bad caller)
    // printf("loadDirectory(): name='%s' isDir=%d blockNumber(logical)=%d\n",
        //dirEntry->name, dirEntry->isDir, dirEntry->blockNumber-1);
    if (!dirEntry){
        printf("Error: LoadDirectory called a NULL DE.\n");
        return NULL;
    }
    if (dirEntry->isDir != 1){
        printf("Error: LoadDirectory called on DE that is not a directory.\n");
        return NULL;
    }
    // Use VCB to get block size.
    struct VCB* v = getVCB();
    int blockSize = v->blockSize;
    // Calculate how many blocks we need to read for this directory.
    int bytesToRead = (int)dirEntry->numBytes;
    int blocksToRead = (bytesToRead + (blockSize - 1)) / blockSize;
    if (blocksToRead <= 0) blocksToRead = 1;
    // Allocate memory for directory contents
    size_t bytesToMalloc = (size_t)blocksToRead * (size_t)blockSize;
    char *buffer = (char *)malloc(bytesToMalloc);
    // Check for malloc failure
    if (!buffer){
        printf("Error: malloc failue in LoadDirectory().\n");
    }
    //
    int lba = dirEntry->blockNumber;
    printf("In loadDirectory(): lbaReading block number : %d\n", lba);
    // Read from disk
    u_int64_t blocksRead = LBAread(buffer, blocksToRead, lba);
    if (blocksRead != blocksToRead) {
        printf("Error: LoadDirectory failed to read %d block (got %lu blocks).\n", blocksToRead, blocksRead);
        free (buffer);
        return NULL;
    }
    return (DE *)buffer;
}
```

loadDirectory() is our function that we use to access the metaDeta of a DE off disk using LBAread(). We first make sure this was called on a directory by checking the flag to see if it marked 1 for dir and 0 for file. We then get blockSize from the VCB to do arithmetic to see how many blocks we need to read. We then take the blocks to read and make a buffer of data so that we can LBAread that correct number making sure to read from the correct block (lba).

(Joe) fs_isFile()

```c
// Joe
// Returns 1 if file, 0 for dir, and -2 if called with bad parameters.
int fs_isFile(char * filename){
    if (strcmp(filename, ".") == 0) return 0;
    if (strcmp(filename, "..") == 0) return 0;
    if (!filename || strlen(filename) == 0) return -2;
    printf("File name in isFile():%s\n", filename);
    DE *cwd = alreadyLoadedCWD;
    int index = findInDirectory(cwd, filename);
    if (cwd[index].isDir == 1)
        return 0;
    return 1;
}
```

fs_isFile() is the function that is called in the shell to determine if the filename is a directory or file. It takes in the file name as a parameter. Does a quick check to return 0 in case name is . or .. because those are always directories. Then it checks for a bad call. Finally we assume since we are just given a file name that we should look for it in the directory we are currently in, get the CWD and look for an entry with a matching name using findInDirectory(). If there is a match and the isDir is 1 we return 0 saying ok this is for sure a file. Else we return -1 because it is the logical complement.

(Joe) fs_isDir()

```
// Joe
// Returns 1 if directory, 0 for file. -1 for error resulting from call with bad pathname.
int fs_isDir(char * pathname){
    if (!pathname || strlen(pathname) == 0) return 0;

    parsePathInfo ppi;
    if (parsePath(pathname, &ppi) != 0){
        printf("Error in fs_isDir: Invalid path : %s \n", pathname);
        return -1;
    }
    return ppi.parent[ppi.index].isDir;
}
```

fs_isDir() is a much simpler version of fs_isFile. Because we know it is being called in the shell with a pathname we can simply use parsePath being careful to add a check for errors. Once we have parsed the path we simply use the ppi struct’s info populated therein to look at the isDir value which should accurately reflect desired return values; 1 for directory 0 for file.

(James) fs_stat()

```c
// James
int fs_stat(char *path, struct fs_stat *buf){
    if(path == NULL || buf == NULL){
        return -1; // Invalid arguments
    }

    parsePathInfo ppi;
    int parseResult = parsePath(path, &ppi);
    if(parseResult != 0){
        return -2; // Path parsing error
    }

    DE *targetDE;
    if(ppi.index >= 0){
        targetDE = &ppi.parent[ppi.index];
    } else {
        return -3; // File or directory not found
    }

    buf->st_size = (off_t)targetDE->numBytes;
    struct VCB* vcb = getVCB();
    buf->st_blksize = (blksize_t)vcb->blockSize;
    buf->st_blocks = (blkcnt_t)((targetDE->numBytes + 511) / 512); // Number of 512B blocks allocated
    buf->st_accesstime = targetDE->accessDate;
    buf->st_modtime = targetDE->modifyDate;
    buf->st_createtime = targetDE->createDate;
    return 0; // Success
}
```

fs_stat() is a function in order to get files or directory by following a structured error-handling approach. First, I validate entering parameters to prevent NULL that would cause segmentation fault. Then, I delegate path parsing to parsePath function, which includes parsePathInfo structure containing the parent directory and target directory entry index. I check if the index is valid to confirm the file exists. For the stat buffer, I copy file size from VCB using getVCB() because it’s a file system property. For block count, I use ceiling division (numBytes + 511)/ 512 to round up partial blocks.

(Joe) fs_setcwd()

```c
// CD - Parse pathname passed in by caller, checks for bad call
// e.g. pathname is invalid, pathname points to non-dir, uses that
// to update global cwd cache.
int fs_setcwd(char *pathname){
    //printf("Path name prior in setCwd():%s\n", pathname);…
    if (!pathname || strlen(pathname) == 0) return 0;
    DE *child = NULL;
    parsePathInfo ppi;
    // Populate ppi struct
    if (parsePath(pathname, &ppi) != 0) {
        printf("Error in fs_setcwd: Invalid path\n");
        return -1;
    }
    // Case : Path refers to parent (eg. '/')
    if (!ppi.lastElementName){
        //path already points to cwd
        alreadyLoadedCWD = ppi.parent;
        return 0;
    }
    // Assign local scope index from ppi struct for safety.
    int index = ppi.index;
    if (index < 0){
        printf("In fs_setcwd: could not resolve lastElementName.\n");
        return -1;
    }
    // Make sure we caller isn't trying to change to non-directory.
    if (ppi.parent[index].isDir != 1){
        printf("fs_setcwd: %s is not a directory.\n", ppi.lastElementName);
        return -1;
    }
    // Case : Final element is dir and exists.
    child = &ppi.parent[index];
    // Load directory metadata off disk.
    DE *newCWD = loadDirectory(child);
    //printf("DEBUG cd: found '%s' at idx=%d, isDir=%d, startLBA=%d size=%d\n",
       //ppi.lastElementName, index, child->isDir,
       //child->blockNumber, child->numBytes);
    if (!newCWD){
        printf("fs_setcwd: failed to load directory '%s'\n", ppi.parent[index].name);
        return -1;
    }
    // Free parent only if parsePath() loaded it dynamically from global cache's.
    if (ppi.parent != alreadyLoadedRootDir && ppi.parent != alreadyLoadedCWD){
        free(ppi.parent);
    }
    // Assign global cache to correct directory.
    alreadyLoadedCWD = newCWD;
    return 0;
}
```

fs_setcwd() is responsible for the cd shell command. It uses parsePath to correctly determine where to look, does checks for bad calls, makes sure we dont cd to a file, then loads directory and sets

global cache; making sure to only do so if caches are not manipulated in parsePath().

THE FOLLOWING ARE HELPERS AND THE DATA STRUCTURE NECESSARY FOR FS_GETCWD()

(Joe) findChildHeadByBlock()

```
// Helper function for use in get cwd. Because we need to walk back UP from cwd.
// We cannot use ppi info but we can use where it ends up pointing to find parents block
// number. That way we can walk "up the tree".
int findChildByHeadBlock(const DE *parent, int entryCount, int childHeadBlock) {
    for (int i = 2; i < entryCount; i++) {                // skip '.' and '..'
        if (parent[i].name[0] == '\0') continue;          // empty slot
        if (parent[i].isDir && parent[i].blockNumber == childHeadBlock) {
            return i;
        }
    }
    return -1;
}
```

findChildHeadByBlock() is a function that we can use to traverse back up the pathname. Because parsepath points to the last element's metaData we cannot use just it. We must use this in conjunction with it. It goes through the directories entries looking for the block number of the parent and returns it (i).

(Joe)initVector() and StringBuilder vector data structure

```
typedef struct stringBuilder{
    char **subStrings;
    // Length of vector
    int size;
    // How many elements vector can hold
    int capacity;
} stringBuilder;
```

```
void initVector(stringBuilder* v){
    v->subStrings = NULL;
    // How many elements vector can hold
    v->capacity = 0;
    // Length of vector.
    v->size = 0;
}
```

initVector() & StringBuilder Because of the complexity innate to dynamically build strings of modular length, needing to determine where put / in between, AND reversal I felt it would be much easier with a vector so I made a vector appropriately called stringBuilder to do all the complex path work necessary. So that’s why I made the stringBuilder struct and it’s necessary to have an initialize function i.e. initVector().

(Joe) vectorAdd()

```c
// Adds token (substring) passed in to vector, returns -1 incorrect call with nulls;
// -2 for realloc failure; -3 for strdup failure.
// Else it returns 0 on success.
int vectorAdd(stringBuilder* v, const char * substring){
    // Check for nulls
    if (!v || !substring) return -1;

    // Grow vector if needed.
    if (v->size == v->capacity){
        // In edge case; 1st step only set initial capacity of 0 to 8.
        // Otherwise double capacity.
        int newCapacity = (v->capacity == 0) ? 8 : v->capacity * 2;
        // Make temp to copy from
        char **tmp = (char **)realloc(v->subStrings, newCapacity * sizeof(*tmp));
        if (!tmp) return -2;

        v->subStrings = tmp;
        v->capacity = newCapacity;
    }
    // Add to vector making sure to increment size while
    v->subStrings[v->size] = strdup(substring);
    if (!v->subStrings[v->size]) return -3;
    v->size++;
    return 0;
}
```

vectorAdd() is a simple add function regular to vectors with additional functionality to return sentinel values based on potential errors and usage of strdup() to copy pathname passed in (substring) to the substrings.

(Joe) build()

```c
// Function for adding / delimiter to properly format pathname string to return.
char * build(const stringBuilder *v, char separator){
    // Null check
    if (!v) return NULL;
    // Empty vector
    if (v->size == 0){
        char *emptyVector = (char *)malloc(1);
        if (!emptyVector) return NULL;
        emptyVector[0]='\0';
        return emptyVector;
    }
    // Get total length = sum of subStrings length + separators + '\0'
    size_t length = 1; //Start at one for atleast '\0'

    // Iterate along vector of substrings adding up their length in bytes.
    for (size_t i = 0; i < v->size; i++) {
        length += strlen(v->subStrings[i]);
    }

    // If we had "home" "joe" "documents"
    // size of the vector would be 3.
    // BUT we need to add separators
    // eg. "home" "/" "joe" "/" "documents
    // so we need two more spaces. i.e. o.g. size - 1
    length += (v->size - 1);

    // Create an out buffer returnString
    // to return with appropriate length
    char * returnString = (char *)malloc(length);
    // Check for malloc error
    if (!returnString) return NULL;
    // Create a string which we will use to iterate along vector's substrings
    // and add sepator: '/'
    // [string]
    // home joe documents
    char * position = returnString;
    for (size_t i = 0; i < v->size; i++){
        // So long as we aren't at the very first substring we add '/' first.
        if (i > 0) *position++ = separator;
        //Get length next substring to copy from.
        size_t ssLength = strlen(v->subStrings[i]);
        // Copy from substing to position string.
        memcpy(position, v->subStrings[i], ssLength);
        // Move the length of last substring forward.
        position += ssLength;
    }
    // Put null at end of position string.
    *position = '\0';
    return returnString;
}
```

build() is the function we use in get_cwd() not that the string is put in proper order. Here we iteratively add the length of all substrings to see how long the vector is. We then correctly adjust to make sure we have enough space for ‘/’s in between sub-strings. We then malloc a return string of correct length. Now that we iterate along a temp string called position adding the separator passed in ‘/’ and iteratively along using the next substrings length (ssLength). We then null terminate it and return the correctly built pathname.

(Joe) freeVector()

```c
// Function for freeing the vector in memory once it's done with use.
void freeVector(stringBuilder *v){
    if (!v) return;
    // Free each substring
    for (size_t i = 0; i < v->size; i++){
        free(v->subStrings[i]);
    }
    // Free substring pointer
    free(v->subStrings);
    // Reset
    v->subStrings = NULL;
    v->size = 0;
    v->capacity = 0;
}
```

freeVector() once we are done with the vector because it was malloc’d we need to make sure we have a clean function to correctly free all the components of subStrings and reset struct’s fields.

(Joe) fs_getcwd()

```c
// Joe
// PWD - Uses a vector to walk from cwd back up to root adding directory names
// to stringBuilder vector. It then reverses path to get names in correct order.
// Finally it puts '/' in between names to fit proper format and returns that string.
char * fs_getcwd(char *pathname, size_t size){
    // Invalid parameter checks.
    if (!pathname || size == 0){
        return NULL;
    }
    if (!alreadyLoadedCWD || !alreadyLoadedRootDir) {
        return NULL;
    }

    // /Home/Documents/HW/csc..filesystem/
    // Edge case: if he CWD is the root, return "/"
    DE * cwd = alreadyLoadedCWD;
    if (cwd[0].blockNumber == cwd[1].blockNumber){
        if (size < 2) return NULL;
        pathname[0] = '/';
        pathname[1] = '\0';
        return pathname;
    }

    stringBuilder parts;
    initVector(&parts);
    DE * currentDir = cwd;
    // Walk back recursively along directory entries tree.
    while (1) {
        DE *parent = loadDirectory(&currentDir[1]);
        if (!parent){
            freeVector(&parts);
            return NULL;
        }
        int index = (int)findChildByHeadBlock(parent, parent[0].numBytes/ sizeof(DE),currentDir[0].blockNumber);
        if( index < 0){
            free(parent);
            freeVector(&parts);
            return NULL;
        }
        if (vectorAdd(&parts, parent[index].name) != 0){
            free(parent);
            freeVector(&parts);
            return NULL;
        }
        // Only instance where "." and ".." share blockNumber will be root.
        // So we know we're done.
        if (parent[0].blockNumber == parent[1].blockNumber){
            free(parent);
            break;
        }
```

(Joe) fs_getcwd()

```
    // Now that we have added to the vector starting from cwd working up to root.
    // The vector's substrings are in reverse order. So now we loop and reverse.
    // Them to get them in our desired order.
    if (parts.size > 1) {
       size_t endIndex = parts.size - 1;
       size_t startIndex = 0;
       while (startIndex < endIndex){
        char *tmp = parts.subStrings[startIndex];
        parts.subStrings[startIndex] = parts.subStrings[endIndex];
        parts.subStrings[endIndex] = tmp;
        startIndex++;
        endIndex--;
       }
    }
    char *returnString = build(&parts, '/');
    // Get length of string to return
    size_t returnLength = strlen(returnString);
    // So we can compare it to sized passed in.
    // '/' "returnStringLength" '/' for instance that's why we add two.
    if (returnLength + 2 > size){
        free(returnString);
        freeVector(&parts);
        return NULL; //Buffer too small
    }
    // Now that we have correct return string and verified sufficient
    // Space and pathname length. We just memcpy to buffer (pathname).
    pathname[0] = '/';
    memcpy(pathname +1, returnString, returnLength + 1);
    // Free everthing now that we are done.
    free(returnString);
    freeVector(&parts);
    return pathname;
}
```

fs_getcwd() is our pwd function. It uses the aforementioned helpers to accomplish the task of starting at CWD, adding names of each dir in path to substrings vector until it reaches root. It knows it has reached root when the current directory's block number matches its parent. Because it walks back up the tree adding substrings the pathname is reversed. So it first uses a tmp string which is set to first substring and reverses them using endIndex (starting before null terminator). It then rebuilds the string with / delimiter using the build() function, appropriately frees vector, null terminates string, and memcpy’s to the caller's pathname and returns the properly formatted string to reflect the path to CWD.

Accessor functions for FSM(); fsmReserveLBA(), fsmReleasLBA(), and fsmIsUsed()

```
// Access functions so other layers can use FSM.
int fsmReserveLBA(int lba){
    if (lba < 0 )
        return -1;
    setBit(lba);
    fsmWrite();
    return 0;
}
int fsmReleaseLBA(int lba){
    if ( lba < 0 )
        return -1;
    clearBit(lba);
    fsmWrite();
    return 0;
}
int fsmIsUsed(int lba){
    if ( lba < 0 )
        return 1;
    return getBit(lba);
}
```

At this point in MS2 we realized our previous approach of keeping functions of FSM wouldn’t work as MFS and initFS layer needed to be able to correctly reflect its changes to FSM. All we had to do was simply make public functions that call the FSM’s static (private) functions.

(Enrique) fs_opendir()

```
// Enrique
// Directory iteration functions
// Opens a directory for reading its contents. This function prepares the
// directory for iteration by loading it into memory or using a cached
// version and creating a directory descriptor fdDirthat tracks our
// position as we read through entries.
fdDir * fs_opendir(char *pathname){
    // We need to check for NULL before calling strlen, which would crash
    // if pathname is NULL. parsePath also checks for NULL, but we need
    // to validate here first since we use strlen before calling parsePath.
    if (pathname == NULL) {
        return NULL;
    }

    // Parse the pathname to break it down into components and find the
    // target directory entry. The parsePathInfo structure will contain
    // information about where the directory is located.
    parsePathInfo ppi;
    int parseResult = parsePath(pathname, &ppi);
    if (parseResult != 0) {
        return NULL; // Path doesn't exist or is invalid
    }

    DE tempDir;
    DE *dirToCopy = NULL;
```

(Enrique) fs_opendir()

```
// When index is -2, it means we're opening the parent directory itself
// like opening "/" for root. This is how parsePath indicates that
// the path refers to a directory rather than something inside it.
if (ppi.index == -2) {
    // This means we're opening the parent directory itself like "/" for root.
    // Root directory should have been loaded in initFileSystem so ppi.parent
    // should point to alreadyLoadedRootDir. If NULL, indicates a
    // root wasn't loaded during initialization.
    if (ppi.parent == NULL) {
        printf("Error: fs_opendir - root directory not loaded. "
               "initFileSystem must be called first.\n");
        return NULL;
    }
    //Clean slate for copy
    memset(&tempDir, 0 , sizeof(DE));
    tempDir.blockNumber = ppi.parent[0].blockNumber;
    tempDir.numBytes = ppi.parent[0].numBytes;
    tempDir.isDir = ppi.parent[0].isDir;
    dirToCopy = &tempDir;
} else if (ppi.index >= 0 && ppi.parent[ppi.index].isDir == 1){
    dirToCopy = &ppi.parent[ppi.index];
} else {
    return NULL;
}
DE *dirOnDisk = loadDirectory(dirToCopy);
if (!dirOnDisk){
    return NULL;
}
// Calculate how many directory entries can fit in this directory.
// The first entry index 0 stores the total size in numBytes, so
// we divide by the size of one directory entry to get the count.
int totalEntries = dirOnDisk[0].numBytes / sizeof(DE);
if (totalEntries <= 0) {
    free(dirOnDisk);
    return NULL; // Invalid directory size
}
```

(Enrique) fs_opendir()

```
    // Allocate memory for the directory descriptor structure. This is
    // like a file descriptor but for directories it keeps track of
    // which entry we're currently looking at.
    fdDir *dirp = (fdDir *)malloc(sizeof(fdDir));
    if (dirp == NULL) {
        // If allocation fails clean up the directory we loaded but only
        // if it's not cached since we shouldn't free cached directories
        free(dirOnDisk);
        return NULL;
    }
    dirp->dirEntryPosition = 0;
    dirp->directory = dirOnDisk;
    dirp->d_reclen = sizeof(DE);
    dirp->totalEntries = totalEntries;
    dirp->di = NULL;
    // Initialize the directory descriptor. We start at position 0 the
    // first entry, point to our loaded directory, and set the record
    // length. The di pointer will be allocated when we first read an
    // entry (lazy allocation to save memory if the directory is never
    // actually read).

    return dirp;
}
```

fs_opendir() The fs_opendir() function opens a directory so you can read through its contents. We’re preparing to read through the list of files and subdirectories that are stored in a directory. When you call this function, you pass in a pathname that tells it which directory you want to open. The function first checks if the pathname is NULL, and if so, it returns NULL. Then it uses a helper function called parsePath() to break down the pathname and figure out where the directory is located in the filesystem. Once the path is parsed, the function needs to figure out which directory entry to load. If the index is -2, that means we're opening the parent directory. In this case, the function checks if the root directory was loaded during initialization, and if not, it returns NULL with an error message. Otherwise, it creates a temporary directory entry with the information from the parent. If the index is valid and the entry is actually a directory, not a file, the function uses that directory entry. If the path doesn't exist or points to something that isn't a directory, the function returns NULL. Next, the function calls loadDirectory() to actually read the directory data from disk into memory. This is important because you can't just access directory information directly from disk—you need to bring it into memory first. If loading fails, the function returns NULL. Once the directory is loaded, the function

calculates how many directory entries can fit in this directory by dividing the total size (stored in the first entry's numBytes field) by the size of one directory entry. This tells us how many slots are available in the directory. Finally, the function allocates memory for a directory descriptor structure (called fdDir). This is like a file descriptor but for directories—it keeps track of which entry we're currently looking at as we iterate through the directory. The function initializes this structure by setting the current position to 0 (the first entry), pointing to the loaded directory data, storing the total number of entries, and setting the record length. It also initializes the directory item info pointer to NULL. This will be allocated later when we actually read an entry. The function returns a pointer to this directory descriptor, which you then use with fs_readdir() to read through the directory entries. If anything goes wrong along the way, the function cleans up any allocated memory and returns NULL.

(Enrique) fs_readdir()

```c
// Returns the next directory entry each time it's called. When there
// are no more entries it returns NULL to signal the end.
struct fs_diriteminfo *fs_readdir(fdDir *dirp){
    // Input validation: need a valid directory descriptor and directory
    if (dirp == NULL || dirp->directory == NULL) {
        return NULL;
    }

    // Use the totalEntries we calculated and stored in opendir. This
    // avoids recalculating it every time readdir is called, which is
    // more efficient

    // Loop through entries starting from our current position. We keep
    // going until we've checked all possible entry slots
    while (dirp->dirEntryPosition < dirp->totalEntries) {
        // Get a pointer to the current directory entry we are examining
        DE *currentEntry = &dirp->directory[dirp->dirEntryPosition];

        // Skip empty entries. When a directory entry is deleted or never
        // used the name field starts with a null terminator. We want to
        // skip these "holes" in the directory and only return valid entries.
        if (currentEntry->name[0] == '\0') {
            dirp->dirEntryPosition++;
            continue;
        }
        // Find the child’s entry in parent by matching the child’s head block.
        // Only allocate the return structure when we
        // actually find a valid entry. This saves memory if someone opens
        // a directory but never reads from it.
        if (dirp->di == NULL) {
            dirp->di = (struct fs_diriteminfo *)
                malloc(sizeof(struct fs_diriteminfo));
            if (dirp->di == NULL) {
                return NULL;
            }
        }
```

```
        // Copy the entrys name into our return structure. We use strncpy
        // to safely copy up to 255 characters leaving room for the null
        // terminator in the 256-character buffer.
        strncpy(dirp->di->d_name, currentEntry->name, 255);
        dirp->di->d_name[255] = '\0'; // Ensure null termination

        // Set the file type based on the isDir flag. This tells the caller
        // whether this entry is a directory can contain other files or
        // a regular file (contains data).
        if (currentEntry->isDir == 1) {
            dirp->di->fileType = FT_DIRECTORY;
        } else {
            dirp->di->fileType = FT_REGFILE;
        }

        // Set the record length to the size of a directory entry. This
        // is useful information for the caller.
        dirp->di->d_reclen = sizeof(DE);

        // Advance our position counter so the next call to readdir will
        // return the next entry in the directory.
        dirp->dirEntryPosition++;

        // Return the information about this entry. Note: we return "." and
        // ".." entries too
        return dirp->di;
    }

    // Reached the end of the directory ... no more entries to return
    return NULL;
}
```

fs_readdir() The fs_readdir() function reads the next entry from a directory that was previously opened with fs_opendir(). Each time you call it, it returns information about the next file or subdirectory in the directory. When there are no more entries, it returns NULL to signal that you've reached the end. When you call this function, you pass in the directory descriptor that you got from fs_opendir(). The function first checks if the descriptor is valid and if it actually points to a loaded directory. If either is NULL, the function returns NULL. The function uses a loop to go through directory entries starting from the current position. It keeps track of where it is using the dirEntryPosition field in the directory descriptor, which was initialized to 0 in fs_opendir(). The loop continues until we've checked all possible entry slots in the directory. As it goes through entries, the function skips empty entries. When a directory entry is deleted or never used, the name field starts with a null terminator. These are like "holes" in the directory, they're slots that exist but

don't contain valid data. The function skips over these and only returns information about entries that actually have names. When the function finds a valid entry, it allocates memory for the return structure (fs_diriteminfo) if it hasn't already been allocated. This structure holds the information about one directory entry that gets returned to the caller. The function only allocates this when it actually finds a valid entry, which saves memory if someone opens a directory but never reads from it. Once the return structure is ready, the function copies the entry's name into it using strncpy() to safely copy up to 255 characters, leaving room for the null terminator in the 256-character buffer. It also ensures the string is null-terminated. The function then sets the file type based on the isDir flag from the directory entry. If isDir is 1, it marks this as a directory (FT_DIRECTORY), which means it can contain other files. Otherwise, it marks it as a regular file (FT_REGFILE), which contains data. This tells the caller whether this entry is a directory or a file. The function also sets the record length to the size of a directory entry, which is useful information for the caller. Then it advances the position counter so the next call to fs_readdir() will return the next entry in the directory. Finally, the function returns a pointer to the directory item info structure containing information about this entry. Note that the function returns "." and ".." entries too, which are special entries that refer to the current directory and parent directory respectively. If the function reaches the end of the directory without finding any more valid entries, it returns NULL to signal that there are no more entries to read.

(Enrique)fs_closedir()

```c
// Closes a directory and frees all resources associated with it.
// Every opendir should have a matching closedir. However, we are
// careful not to free cached directories (root and CWD)
// since they're shared and may still be in use.
int fs_closedir(fdDir *dirp){
    // Input validation: can't close something that doesn't exist
    if (dirp == NULL) {
        return -1;
    }

    // Check if this directory is one of our cached directories. Cached
    // directories root and current working directory are kept in memory
    // for performance and may be used by multiple operations. We should
    // never free them here because other parts of the code might still
    // be using them.
    int isCached = 0;
    if (dirp->directory == alreadyLoadedRootDir ||
        dirp->directory == alreadyLoadedCWD) {
        isCached = 1;
    }

    // Only free the directory data if its not cached. If its cached,
    // we leave it in memory.
    if (!isCached && dirp->directory != NULL) {
        free(dirp->directory);
    }

    // Free the directory item info structure if it was allocated during
    // readdir operations. This structure holds information about individual
    // entries that we return to the caller.
    if (dirp->di != NULL) {
        free(dirp->di);
    }

    // Free the directory descriptor structure itself. This was
    // allocated in opendir and tracks our iteration state.
    free(dirp);

    return 0;
}
```

fs_closedir() The fs_closedir() function closes a directory that was previously opened with fs_opendir() and frees all the memory that was allocated for it. When you call this function, you pass in the directory descriptor that you got from fs_opendir(). The function first checks if the descriptor is NULL, and if so, it returns an error code since you can't close something that doesn't exist. The function only frees the directory data if it's not cached. If the directory is cached, it leaves it in memory so other parts of the code can continue

using it. Next, the function frees the directory item info structure (di) if it was allocated during fs_readdir() operations. Finally, the function frees the directory descriptor structure itself. This structure was allocated in fs_opendir() and tracks the iteration state like which entry we're currently looking at. After freeing it, the function returns 0 to indicate success.

Descriptions Of Functions MS 3

(Ian, Joe) Touch: Going into milestone three, we were all conceptually trying to make sense of the separation between the mfs layer and the io layer, and how calls are going to be made, and how information is going to be accessed between them. Based on this, we came up with the idea of this Touch() helper function.

During the chain of function calls, this touch helper still ended up being an integral part of how new empty files are made in our filesystem. The chain of calls is as follows: first cmd_touch -> b_open. During the run of b_open, the fileInfo object is extracted using a getFileInfo call. During this getFileInfo call, the Touch helper function helper is called and the return value from which is used as the location which will be used to populate the fileInfo struct. In summary: the chain of calls to the helper when a user enters the touch command in the fs prompt is : cmd_touch -> b_open -> getFileInfo -> Touch

Due to the chain of calls described above, Touch helper is only called when the following conditions have been met in the functions above: the file being referenced by b_open does not exist and the flags are appropriate from the io layer to create a file.

Given that the above is true, the job of the helper is to create a new empty file with the given name, and appropriately update the parent to include a directory entry describing the new file.

Touch first needs the reference to our freeTail tracker to know where it is appropriate to write. With that, Touch then finds a free DE slot in the parent (parent is passed in as arg) and populates a new DE at that index in the parent. It populates all info accordingly, setting numBytes to 0 because the file will be empty. After that touch simply updates the vcb tracker info: freeTail and numFreeBlocks to reflect a new addition. Finally, LBAwrite is called for one block at the parent location.

(Ian, Joe, Enrique) b_read: Our approach to b_read was strongly influenced from what we had learned about buffered reads from Assignment 5. We took that understanding and modeled it to work as the main way data will be read in our filesystem.

Similar to assignment 5, the buffered approach lets us only LBAread when absolutely necessary. This meant checking (in the same way as HW5) that the data isn’t already in our buffer.

The main read loop only ends when the amount of bytes read matches the amount specified by the caller’s count of bytes they want to read.

By keeping track of currentFilePosition, bufferStartPosition, and numValidBufferBytes, we could ensure that we never LBAread unless the currentFilePosition is outside the bounds of the buffer, meaning the data is in fact not immediately accessible. This is the optimization that professor Bierman discussed in class.

These variables also allowed us to calculate the bytesToCopy in a way that ensures we never copy beyond the bounds specified by the caller, i.e. unwanted data.

(Ian, Joe) b_open: Inspired by assignment 5, our approach to b_open makes use of getFileInfo and the fileInfo struct (see below). This allows us to get needed info from the mfs layer into the io layer. This function will provide the setup for all io layer operations.

Using a free FCB, open first calls getFileInfo which will determine if the file exists already or not and will handle creation if necessary. The fileInfo returned from this, either from an existing or new file, contains the metadata from the mfs layer which will populate the fcb. In addition, open will allocate a buffer for future read/write operations, and set the buffer management related variables to their initial values (0). The provided flags are stored in the FCB as well.

With the FCB initialized, that index is returned to the caller which can be used for any operations needed until the file is closed.

(Ian, Joe) b_close: Using the provided file descriptor, close will first verify that the FCB for it is valid and open. If there is any remaining data in its buffer, it is flushed by writing the partial block to disk.

Close then makes use of updateFileSizeOnDisk (see below) which in turn calls ensureAllocation (see below). This process essentially ensures that the directory entry’s file size is correct and that any used but unmarked blocks are allocated.

Finally, the FCB is reset for future use by open.

(Ian, Joe) getFileInfo and fileInfo struct: The fileInfo struct was our way of filling in the gaps for the io layer, inspired by assignment 5. This is our main bridge between the mfs layer and the io layer. It contains members fileName, fileSize, location, and allocatedBlocks.

The getFileInfo function serves as the way to gather information in the mfs from the io layer, and also serves as a way to create new files and fileInfos if they don’t exist yet. If the file does not exist and the O_CREAT flag is set, the touch helper function (see above) is called to create the file. If the file already exits, it simply gathers the data from the pre-existing parent’s DE entry about that file and uses it to populate the fileInfo.

After that, assuming no errors, the getFileInfo function now hands this fileInfo back up to the io layer where it was called for either the pre-existing file or the now newly created one.

(Ian, Joe) mv shell implementation: We realized that essentially, the mv operates as a cp followed by a rm, basically just cp with one extra step! And good news, we’ve already implemented both of these already! So this was a pretty easy implementation, using our

already done cp and rm. This function creates char star vectors of sizes 3 and 2 for the args to be passed into cp and rm respectively. This essentially allows us to construct a pre-made command line call to these functions using the inputted sourceFilename (inputted to mv as argvec[1]) and destFilename (inputted to mv as argvec[2]). With those, simply call cp followed by mv!

(Ian, Joe) ensureAllocation: This function took many forms as we ironed out the freeTail implementation. This function started out as a part of Joe’s proposed solution to the chicken/egg problem (see above) but eventually was molded into its final form: as a way to check if a grouping of contiguous blocks are marked as used or not. This function is called during b_close, which helps determine what action should be taken when the file closes. If the blocks passed in are already allocated, then no fsm updates or freeTail movement will take place. This means that when a file is being written, this function will be called and allocate the blocks and update freetail. On the other hand, this means that during a read (such as when the user wants to cat a file), ensureAllocation will not alter the freeTail pointer NOR the fsm, allowing for a read that does not disrupt our freetail/fsm system.

(Joe) updateFileSizeOnDisk()

```c
// Function for io layer to keep metaData in FSM, VCB, and MFS layer in sync and on disk.
int updateFileSizeOnDisk(char *pathname, int newSize, int lba){
    if (!pathname)
        return -1;
    parsePathInfo ppi;
    // Point to appropriate file
    int ppResult = parsePath(pathname, &ppi);
    if (ppResult != 0){
        printf("In updateFileSizeOnDisk: parsepath called with invalid path.\n");
        return -2;
    }
    if (ppi.index < 0){
        printf("In updateFileSizeOnDisk: DE not found for:%s\n", pathname);
        return -3;
    }
    // Get previous size to determine allocation needs
    int OldSize = ppi.parent[ppi.index].numBytes;
    // Update Size and Timestamp
    ppi.parent[ppi.index].numBytes = newSize;
    ppi.parent[ppi.index].modifyDate = time(NULL);
    struct VCB *vcb = getVCB();
    // Do arithmetic to see if we went beyond one block initial allocation of DE's
    int blockSize = vcb->blockSize;
    // Get number of blocks to verify in bitmap.
    int oldBlocks = (OldSize + blockSize - 1) / blockSize;
    int newBlocks = (newSize + blockSize - 1) / blockSize;
    // Case we need to reflect growth in FSM.
    if (newBlocks > oldBlocks){
        int blocksToCheck = newBlocks;
        int blocksToAllocate = ensureAllocation(blocksToCheck, lba);
        if (blocksToAllocate >0){
            printf("In updateFileSize on disk: %d new blocks allocated for filename: %s\n"
        , blocksToAllocate, ppi.lastElementName);
        }
    } else if ( newBlocks < oldBlocks){
        // Case we need to reflect shrinkage in FSM
        int blocksToFree = oldBlocks - newBlocks;
        int firstFreeLBA = lba + newBlocks;
        int lastFreeLBA = lba + oldBlocks - 1;
        int blocksFreed = 0;
        // Deallocate loop
        for (int i = firstFreeLBA; i <= lastFreeLBA; i++){
            if(fsmIsUsed(i)){
                fsmReleaseLBA(i);
                vcb->numFreeBlocks++;
                blocksFreed++;
            }
        }
        // Update tail if optimization condition is met.
        int tailLocation = vcb->freeTail;
        if(lastFreeLBA == tailLocation-1){
            vcb->freeTail -= blocksToFree;
            printf("In updateFileSizeOnDisk just deallocated %d blocks, Tail now set to %d",
            blocksFreed, vcb->freeTail);
        }
        // Save updated FSM Changes to disk.
        fsmWrite();
    }
    // Persist metadata changes of DE to disk.
    LBAwrite(ppi.parent, 1, ppi.parent[0].blockNumber);
    return 0;
}
```

updateFileSizeOnDisk() because the io layer does not need to be exposed to any of the other components e.g. FSM, VCB, MFS but it uses meta data to write; if things change it must have a way to update the metaData and make sure those changes persist on disk. Here we created this function to be called in b_close() (i.e. where changes would be

finalized) to talk *back up* to the MFS layer to make sure everything is in sync. It is implemented inside the MFS layer but only the function prototype is exposed to the io layer via its placement in fileInfo.h. That way it can call parsePath to find the appropriate file, update its new byte count and modify the date. It then checks to see if bits in FSM are correctly reflected given a modification in file size making changes ONLY when a shrinkage/growth occurs such that FSM is incorrect and makes necessary changes.

(Joe) b_write()

```c
// Interface to write function
int b_write (b_io_fd fd, char * buffer, int count)
    {
    // TO DO
    // CHECK FOR PROPER FLAGS
    // MAKE SURE SEEK IS PUT IF AND WHERE NECESSARY
    if (!buffer || count <= 0)
        return -2; // Bad call
    if (startup == 0) b_init();  //Initialize our system
    // Step 1: Validate fd.
    // check that fd is between 0 and (MAXFCBS-1)
    if ((fd < 0) || (fd >= MAXFCBS))
        {
        return (-1);                    //invalid file descriptor
        }
    // Step 2: Get pointer to the file's FCB entry;
    b_fcb *fcb = &fcbArray[fd];
    if (!fcb->fi)
        return -3; // Bad file info
    if (fcb->flags == O_RDONLY){
        printf("In b_write() : file contain's O_RDONLY Flag: aborting now.\n");
        return -1;
    }
    // Step 3: Check for Append flag.
    // Move the buffer forward to end of file.
    if (fcb->flags & O_APPEND)
        fcb->currentFilePosition = fcb->fi->fileSize;
    // Step 4: Write Loop
    // Write until we reach number (count) passed in.
    size_t bytesWritten = 0;
    while (bytesWritten < count){
        // Step 5: Check to see if current file position
        // is in the current buffer window.
        // Our file pointer is behind the start of block              // Is our file position ahead of buffer
        // file starts at 700 ; buf[0] = byte 1024; WRONG FLUSH       // buf[0] = byte 0; file starts at 513; WRONG FLUSH
        // In both this cases buffer does NOT contain byte we want to write so we mush flush.
        int outsideWindow = (fcb->currentFilePosition < fcb->bufferStartPosition ||
        fcb->currentFilePosition >= fcb->bufferStartPosition + BLOCK_SIZE);

        if(outsideWindow && fcb->numValidBufferBytes > 0){
            // Flush buffer
            int blockIndex = fcb->bufferStartPosition / BLOCK_SIZE;
            int previousLocation = fcb->fi->location;
            blockIndex +=  previousLocation;
            LBAwrite(fcb->buf, 1, blockIndex);
            fcb->numValidBufferBytes = 0;
        }
```

b_write()

```
        if (outsideWindow && fcb->numValidBufferBytes == 0){
            // Realign the buffer
            fcb->bufferStartPosition = (fcb->currentFilePosition / BLOCK_SIZE) * BLOCK_SIZE;

            if (fcb->bufferStartPosition < fcb->fi->fileSize){
                // There is existing data in this block so we need to read it.
                int blockIndex = fcb->bufferStartPosition / BLOCK_SIZE;
                LBAread(fcb->buf, 1, fcb->fi->location + blockIndex);
                int remainingFileBytes = fcb->fi->fileSize - fcb->bufferStartPosition;
                fcb->numValidBufferBytes = (remainingFileBytes < BLOCK_SIZE) ? remainingFileBytes : BLOCK_SIZE;
            } else {
                // We're righting past EOF so we need to start with a 0'd block
                memset(fcb->buf, 0, BLOCK_SIZE);
                fcb->numValidBufferBytes = 0;
            }
        }
        // Step 6: Compute current offset.
        size_t offset = fcb->currentFilePosition - fcb->bufferStartPosition;
        int spaceInBlock = BLOCK_SIZE - offset;

        // Step 7: Determine number of bytes we can copy into buffer before end of block.
        int remaining = count - bytesWritten;
        int bytesToCopy = (remaining < spaceInBlock) ? remaining : spaceInBlock;

        // Step 8 : Update values
        // Take the minimum of space in block if only one block is left.
        // else use total amount of bytes left to copy
        memcpy(fcb->buf + offset, buffer + bytesWritten, bytesToCopy);
        fcb->currentFilePosition += bytesToCopy;
        bytesWritten += bytesToCopy;
        // Take the max of current valid bytes or offset + bytestocopy
        fcb->numValidBufferBytes = fcb->numValidBufferBytes > offset + bytesToCopy ? fcb->numValidBufferBytes : offset + bytesToCopy;
        // Step 9: Check for full buffer. If full flush
        int blockIndex = fcb->bufferStartPosition / BLOCK_SIZE;
        if (fcb->numValidBufferBytes == BLOCK_SIZE || offset + bytesToCopy == BLOCK_SIZE){
            int previousLocation = fcb->fi->location;
            blockIndex += previousLocation;
            LBAwrite(fcb->buf, 1, blockIndex);
            fcb->numValidBufferBytes=0;
        }
        // Step 10: If writing past old file size
        // Update fileInfo correctly
        if (fcb->currentFilePosition > fcb->fi->fileSize){
            fcb->fi->fileSize = fcb->currentFilePosition;
        }
    }
    return bytesWritten;
}
```

b_write() is similar to read with only difference in fact that it writes to the caller’s buffer as opposed to just reading from. For that reason it seems more complicated than it actually is because of the many additional conditional checks and necessary adjustments not found in read. Fundamentally it is simply a way to make sure that you are correctly incrementally writing *boats* of data chunks. It makes sure to check the *boat* never gets out of position, you only send it when it’s full, *flush* it on send, wash, rinse and repeat, making sure to update appropriate info as needed. It then returns the number of bytesWritten that it increments.

(Enrique)b_seek()

```
// GIVEN PROPER FLAG CONDITIONS
// Interface to seek function
// This function moves the file position pointer to a new location in the file
int b_seek (b_io_fd fd, off_t offset, int whence)
    {
    if (startup == 0) b_init();  //Initialize our system

    // check that fd is between 0 and (MAXFCBS-1)
    if ((fd < 0) || (fd >= MAXFCBS))
        {
        return (-1);                    //invalid file descriptor
        }

    // Also need to make sure this file is actually open
    // If fi is NULL, then this file descriptor isn't being used
    if (fcbArray[fd].fi == NULL)
        {
        return (-1);                    // File isn't open, can't seek
        }

    // Get a pointer to the file control block so I can access the file's info
    b_fcb *fcb = &fcbArray[fd];
    off_t newPosition;  // This will hold where we want to move to
```

```
// Now I need to figure out the new position based on whence
// whence tells us where to start counting from, and offset tells us how far
switch (whence)
    {
    case SEEK_SET:
        // SEEK_SET means start from the beginning of the file
        // So the new position is just the offset value
        newPosition = offset;
        break;
    case SEEK_CUR:
        // SEEK_CUR means start from where we currently are
        // So add the offset to our current position
        // Offset can be positive (move forward) or negative (move backward)
        newPosition = fcb->currentFilePosition + offset;
        break;
    case SEEK_END:
        // SEEK_END means start from the end of the file
        // So add offset to the file size
        // Offset 0 would put us at the end, negative would go back from end
        newPosition = fcb->fi->fileSize + offset;
        break;
    default:
        // If whence isn't one of the three valid options, that's an error
        return (-1);
    }

// Can't seek to a negative position - that would be before the file starts
if (newPosition < 0)
    {
    return (-1);                    // Invalid position
    }

// Now I need to check if we're moving outside the current buffer window
// The buffer only holds one block (512 bytes) at a time
// If we're seeking to a different block, we need to handle the buffer
int outsideWindow = (newPosition < fcb->bufferStartPosition ||
                     newPosition >= fcb->bufferStartPosition + BLOCK_SIZE);
```

```
// If we're leaving the current buffer window, we need to deal with the buffer
if (outsideWindow)
    {
    // If there's data in the buffer that hasn't been written to disk yet,
    // we need to flush it first or we'll lose it
    if (fcb->numValidBufferBytes > 0)
        {
        // Figure out which block on disk this buffer corresponds to
        // bufferStartPosition tells us where in the file the buffer starts
        // Divide by BLOCK_SIZE to get which block number that is
        int blockIndex = fcb->bufferStartPosition / BLOCK_SIZE;
        // Add the file's starting location to get the actual disk block
        blockIndex += fcb->fi->location;
        // Write the buffer to disk so we don't lose any changes
        LBAwrite(fcb->buf, 1, blockIndex);
        }
    // Mark the buffer as empty/invalid so the next read or write will
    // know it needs to load the correct block for the new position
    fcb->numValidBufferBytes = 0;
    }

// Update the current file position to where we're seeking to
fcb->currentFilePosition = newPosition;

// Return the new position
return (newPosition);
}
```

b_seek() The b_seek() function moves the file position pointer to a new location in a file that's already open. This is useful when you want to read from or write to a specific part of the file without having to read through everything that comes before it. When you call this function, you pass in three parameters: a file descriptor, an offset, and a whence value. The function first checks if the system has been initialized, and if not, it initializes it. Then it validates that the file descriptor is within the valid range and that the file is actually open. If the file descriptor is invalid or the file isn't open, the function returns -1. The function uses a switch statement to figure out the new position based on the whence parameter. There are three possible values: SEEK_SET, SEEK_CUR, and SEEK_END.

If whence is SEEK_SET, that means the offset is specified from the beginning of the file. If whence is SEEK_CUR, that means the offset is from the current position. The offset can be positive or negative. One common use case is SEEK_CUR with an offset of 0, which means "don't move, just tell me where I am." This is used a lot to check the current position in the file without actually moving the pointer. If whence is SEEK_END, that means the offset is from the end of the file. So SEEK_END with an offset of 0 would position you at the end of the file. This is what append mode does—it sets the file

position to the end of the file. If you read from the end of the file, you'll get an end-of-file indication because there's nothing left to read. But if you write from the end, you're continuing to write and appending new data to the file. If whence isn't one of these three valid options, the function returns -1. After calculating the new position, the function checks if it's negative. You can't seek to a negative position because that would be before the file starts, so if the new position is negative, the function returns -1.

The filesystem uses buffering, which means it only keeps one block (512 bytes) of the file in memory at a time. If we're seeking to a position that's outside the current buffer window, we need to deal with the buffer first. The function checks if the new position is before the buffer's start position or after the buffer's end position. If we're leaving the current buffer window, the function needs to handle any unwritten data. If there's data in the buffer that hasn't been written to disk yet, we need to flush it first or we'll lose those changes. To do this, the function figures out which block on disk this buffer corresponds to by dividing the buffer's start position by the block size, then adds the file's starting location to get the actual disk block number. It then writes the buffer to disk using LBAwrite() to make sure we don't lose any changes.

After flushing any unwritten data (or if there wasn't any), the function marks the buffer as empty or invalid. This tells the next read or write operation that it needs to load the correct block for the new position, since the current buffer no longer matches where we are in the file. Finally, the function updates the current file position to the new position we calculated, and returns the resulting position in the file. The return value from b_seek() is always the resulting position, so if you do SEEK_CUR with an offset of 0, you get back your current position without moving. This is useful because it tells you "go from here, offset 0, so I'm still here, and here's where I am." If everything worked correctly, you get back the byte position you're now at in the file. If anything went wrong i.e.: invalid file descriptor, file not open, invalid whence, negative position, the function returns -1 to indicate an error.

MS 1 Tables

Responsibilities: MS 1

Joe	Ian	James	Enrique
- Set up writeup for MS 1 - DE struct - createDir() - writeDirToDisk() - initFileSystem()	- fsmWrite() - allocate() - initFSM() - setBit() - clearBit() - getBit()	- DE Struct - createDir() - writeDirToDisk()	- VCB struct - initVCB() - readVCB() - writeVCB() - getVCB() - initFileSystem()

Who Did What: MS 1

Joe	Ian	James	Enrique
- Set up writeup for MS 1 - loadFSM() - DE struct - createDir() - writeDirToDisk() - initFileSystem()	- fsmWrite() - allocate() - initFSM() - setBit() - clearBit() - getBit()	- DE Struct - createDir() - writeDirToDisk()	- VCB struct - initVCB() - readVCB() - writeVCB() - getVCB() - initFileSystem()

MS 2 Tables

Responsibilities: MS 2

Joe	Ian	James	Enrique
- Set up writeup for MS 2 - loadDirectory() - fs_isFile() - fs_isDir()	- parsePath() - fs_mkDir() - fs_delete() - fs_rmdir()	- Implement Linked List Data Structure - Modify DE struct to accommodate linkage - fs_setcwd() - fs_getcwd() - fs_stat()	- findInDirectory() - fs_opendir() - fs_readdir() - fs_closedir()

Who Did What: MS 2

Joe	Ian	James	Enrique
- Setup writeup for MS 2 - loadDirectory() - fs_isFile() - fs_isDir() - fs_setcwd() - fs_getcwd() - fsmReserveLBA() - fsmReleaseLBA() - fsmIsUsed() - initFileSystem() - findChildByHeadBlock() - StringBuilder Vector - initVector() - vectorAdd() - build() - freeVector() - fs_mkdir()	- parsePath() - fs_mkDir() - fs_rmdir() - findInDirectory()	- fsStat()	- fs_opendir() - fs_readdir() - fs_closedir() - initFileSystem()

MS 3 Tables

Responsibilities: MS 3

Joe	Ian	James	Enrique
Touch Helper - b_open () - b_read () - b_write () - b_close ()	Touch Helper - b_open () - b_read () - b_write () - b_close ()	- cp2fs - cp2l Addendum: Help with catching up other’s portions of writeup descriptions.	File Functions - b_open () - b_read () - b_write () - b_seek () - b_close () - handling of flag logic in i/o layer

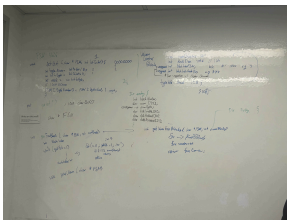
Who Did What: MS 3

Joe	Ian	James	Enrique
- touch() - b_open() - fileinfo.h - getFileInfo() - b_close() - b_read() - mv shell implementation - fs_isDir fixes - b_write() - updateFileSizeOnDisk() - ensureAllocation() - flag handling	- touch() - b_open() - fileinfo.h - getFileInfo() - b_close() - b_read() - mv shell implementation - fs_isDir fixes - ensureAllocation()		- b_open() - fileinfo.h - getFileInfo - b_read () - b_write () - b_seek ()

* A note about github commits…

The commits for milestone 3 are almost exclusively made by josephb333 (Joe). The work in many of these commits was the result of collaborative work via pair programming either remote or in person. These collaborative sessions were between Ian and Joe. Thus the table above is a much more accurate breakdown of contributions.

Meeting Table:

Date	Attended	Purpose	Notes:
10/17	All	First thoughts of the project, how are we going to approach and what functions are we using	 ^Whiteboard notes
10/27	All	Coordinate who is working on what, and when testing should begin.	By Friday we should all be done writing code and have a group meeting during which we begin testing
10/31	All	Get everyone’s individuals’ parts functional to the point we could get a Hexdump.	Finished implementation and fixed all bugs to the point we got working hexdump. Still need to do: 1.Make sure formatted header is included with correct names, descriptions, student ids in ALL files. 2. Put in all of our issues and resolutions 3. Put in hexdump including analysis

11/2	All	Verify hex dump is correct/Resolve subsequent issues.	Finished Correcting Hex Dump. TO do's for tomorrow's meeting at 12pm. 1. Have all code done, including proper headers 2. Put correct hex dump 3. Clarify analysis of hex dumps 4. Put student ids at header of writeup 5. Put ALL of our issues and resolutions (try to include as many pertinent pictures as possible) Objective of meeting. Make sure writeup and code is good to go, push everything, and submit
11/3	All	Final checks	Took much more work than expected. More or longer meetings may be required in future. To Do’s: 1. Familiarize ourselves with necessary next steps so we can appropriately delegate future work for milestones. 2. Submit final writeup once everyone is finished.
11/7	All	Finish all individual functions and start testing in our shell.	80% of functions complete, some missing, others incomplete in functionality. To Do’s: 1. **Enrique** - finish functions and push. 2. **James** - add missing functions to linked list,

			fix usage of FSM. 3. **Joe** - Finish getcwd, necessary vector/stack, fix typos in other parts of code. 4. **Ian** - finish mkdir, ***optionally*** rmdir() too.
11/8	All	Start testing in shell and debugging.	All functions code done. But only pwd and ls working as expected. - md : mkdir() works but is not writing to disk correctly. - cd : set_cwd() is not working at all. - InitFileSystem() has regressed and is not initializing correctly. - ls : works perfect. - pwd : works as expected but needs further testing after we get cd to work.

Meeting Table Continued:

Date	Attended	Purpose	Notes:
11/10	All	Try and implement discontiguous allocation using James’s Linked List.	Between meetings we were able to fix everything and get all mandatory functions for MS 2 working. But only when doing so contiguously as we were still waiting on James’s Linked List to be implemented into our Filesystem correctly. Decision was made to keep working on rmDir contiguously until we could get discontig to work.
11/14	Everyone	Continue Work On MS3. Figure out the best roadmap	Because we incorrectly assumed the same scheme

	Except James	forward.	from MS2 of MFS layer functions being called for shell commands we had to pivot and redivide work after noticing all shell commands for MS 3 were simply calls of IO layer functions. We initially errantly gave all to Enrique. So as to correct mistakenly giving all of MS3 responsibility to one person. The call was made for everyone in the meeting to work on the IO layer together over the weekend until we heard back from James.
11/17	All	Further organizational adjustments. Plan to stress test.	Because we hadn’t heard back from James until Monday's meeting, and due to the nature of the fact that we were able to finish way faster than we first thought. We tried to come up with a way that James could contribute. The call was made to let James help us do a lot of catch up for the final writeup because we all had been lagging when it came to specifically including functions descriptions in writeup, and inclusion of them in proper file headers.

11/18 *Impromptu after class	All	Final readjustment of writeup and delegation of fixes.	- In class we discovered our initial assumption that since b_seek() was never called in the shell we didn’t actually need it. Furthermore, our handling of flags was incomplete to include the need for seek() given appropriate flag conditions. - The call was made to entrust Enrique with finishing up our IO layer with proper flag handling and implementation of seek(). - In the discord, James had mentioned that he would need at least a full two weeks to complete descriptions in the writeup. Given the fact we were 95%+ done and felt like we could finish our respective parts to be ready to submit on Friday. The other members of the group came to a consensus, and the call was made to regress to the initial setup such that everyone took care of the descriptions for what they did.
11/21	ALL	Final polish, checks, and submission	- Allocation was discovered to only be handling growth instances. - Enrique’s changes to our io layer were found to be incorrect so Joe was delegated to fix them. - Final submission pushed back to tomorrow.
11/22	All	Final polish, checks, and submission	- Ian and James are done with his components and portions of writeup. - Joe finished flag handling but still needed to test

		PART TWO	- Joe and Enrique still needed to finish their portions of writeup. - Submission tomorrow pending Joe & Enrique

Compilation & Initial Run

```
student@student:~/csc415-filesystem-josephb333$ make clean
rm fsshell.o fsInit.o b_io.o VCB.o DirectoryEntry.o FreeSpaceMap.o mfs.o vector.o  fsshell
student@student:~/csc415-filesystem-josephb333$ make run
gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -c -o VCB.o VCB.c -g -I.
gcc -c -o DirectoryEntry.o DirectoryEntry.c -g -I.
gcc -c -o FreeSpaceMap.o FreeSpaceMap.c -g -I.
gcc -c -o mfs.o mfs.c -g -I.
gcc -c -o vector.o vector.c -g -I.
gcc -o fsshell fsshell.o fsInit.o b_io.o VCB.o DirectoryEntry.o FreeSpaceMap.o mfs.o vector.
o fsLow.o -g -I. -lm -l readline -l pthread
./fsshell SampleVolume 10000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 bytes.
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
New filesystem detected. Initializing VCB...
VCB initialized: MagicNo=0x41534442, numBlocks=19531, blockSize=512, FSM=1, RootDir=6, NumFr
eeBlocks=19524
writeDirToDisk: writing : 1 blocks to LBA: 6
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    ON    |
| rm                   |    ON    |
| cp                   |    ON    |
| mv                   |    ON    |
| cp2fs                |    ON    |
| cp2l                 |    ON    |
|---------------------------------|
Prompt >
```

Screenshots of Commands (Empty cases)

md (empty)

```
Prompt > md dir1
In mkdir(): Tail is now set to lba number: 8
writeDirToDisk: writing : 1 blocks to LBA: 7
writeDirToDisk: writing : 1 blocks to LBA: 6
Prompt >
```

cd

```
Prompt > md dir1
In mkdir(): Tail is now set to lba number: 8
writeDirToDisk: writing : 1 blocks to LBA: 7
writeDirToDisk: writing : 1 blocks to LBA: 6
Prompt > cd dir1
In loadDirectory(): lbaReading block number : 7
Prompt >
```

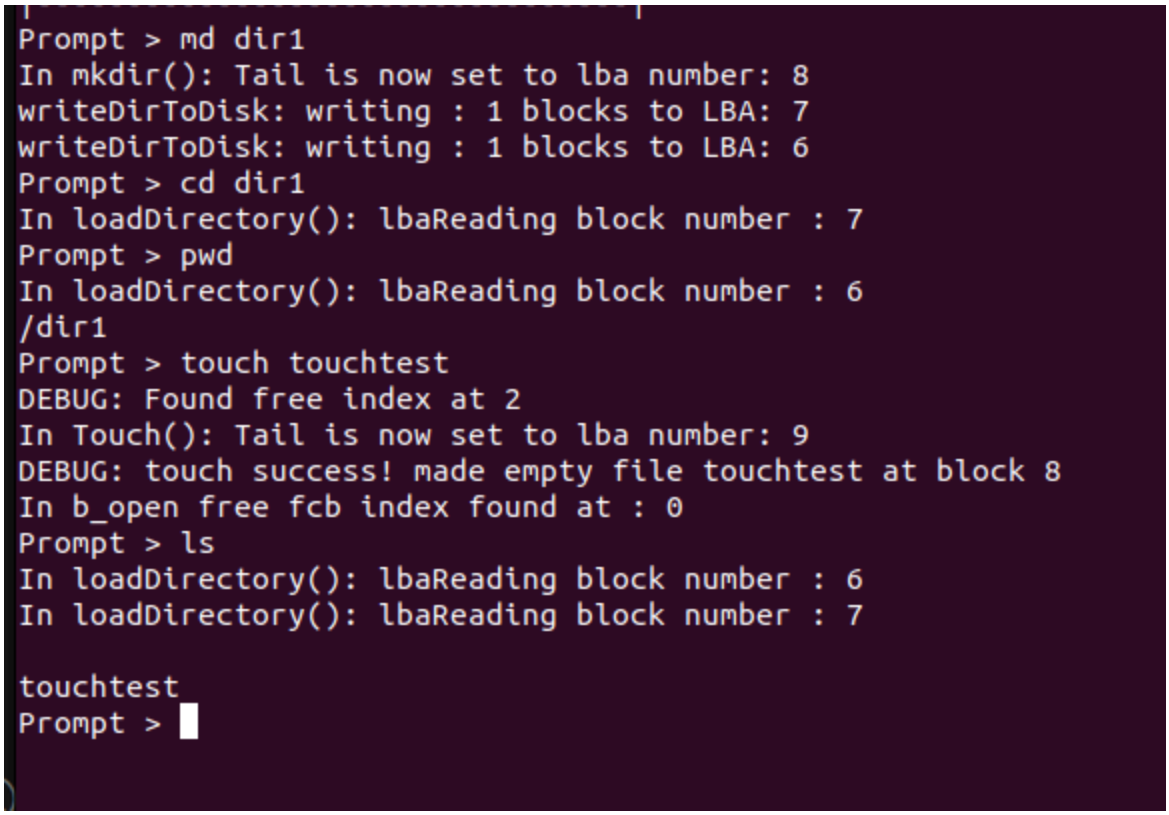
pwd

```
Prompt > md dir1
In mkdir(): Tail is now set to lba number: 8
writeDirToDisk: writing : 1 blocks to LBA: 7
writeDirToDisk: writing : 1 blocks to LBA: 6
Prompt > cd dir1
In loadDirectory(): lbaReading block number : 7
Prompt > pwd
In loadDirectory(): lbaReading block number : 6
/dir1
Prompt >
```

Touch

```
Prompt > pwd
In loadDirectory(): lbaReading block number : 6
/dir1
Prompt > touch touchtest
DEBUG: Found free index at 2
In Touch(): Tail is now set to lba number: 9
DEBUG: touch success! made empty file touchtest at block 8
In b_open free fcb index found at : 0
Prompt >
```

ls



mv (empty file)

```
Prompt > ls
In loadDirectory(): lbaReading block number : 6
In loadDirectory(): lbaReading block number : 7

touchtest
Prompt > mv touchtest mvtest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
DEBUG: Found free index at 3
In Touch(): Tail is now set to lba number: 10
DEBUG: touch success! made empty file mvtest at block 9
In b_open free fcb index found at : 1
File name in isFile():touchtest
In fs_delete going to delete filename:touchtest
In fs_delete going to delete number of blocks: 1
In fs_delete(): successfully delete singleblock at: 8
writeDirToDisk: writing : 1 blocks to LBA: 7
In fs_delete(): Tail is now set to lba number: 10
Prompt > ls
In loadDirectory(): lbaReading block number : 6
In loadDirectory(): lbaReading block number : 7

mvtest
Prompt >
```

cp (empty file)

```
Prompt > ls
In loadDirectory(): lbaReading block number : 6
In loadDirectory(): lbaReading block number : 7

mvtest
Prompt > cp mvtest cptest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
DEBUG: Found free index at 2
In Touch(): Tail is now set to lba number: 11
DEBUG: touch success! made empty file cptest at block 10
In b_open free fcb index found at : 1
Prompt > ls
In loadDirectory(): lbaReading block number : 6
In loadDirectory(): lbaReading block number : 7

cptest
mvtest
Prompt >
```

cat (empty file)

```
Prompt > ls
In loadDirectory(): lbaReading block number : 6
In loadDirectory(): lbaReading block number : 7

cptest
mvtest
Prompt > cat cptest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
Prompt >
```

cp2l (empty file)

```
Prompt > touch cp2ltest
DEBUG: Found free index at 4
In Touch(): Tail is now set to lba number: 12
DEBUG: touch success! made empty file cp2ltest at block 11
In b_open free fcb index found at : 0
Prompt > cp2l cp2ltest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
Prompt >
```

```
student@student:~/csc415-filesystem-josephb333$ ls
 b_io.c                              FreeSpaceMap.c   fsshell.c    SampleVolume
 b_io.h                              FreeSpaceMap.o   fsshell.o    shrek
 b_io.o                              fsInit.c         HEREISFILE   VCB.c
 cp2ltest                            fsInit.o         Hexdump      VCB.h
 DE.h                                fsLow.h          Makefile     VCB.o
 DirectoryEntry.c                    fsLowM1.o        mfs.c        vector.c
 DirectoryEntry.o                    fsLow.o          mfs.h        vector.h
"Don't Worry We Don't Byte - MS1.pdf" FSM.h            mfs.o        vector.o
 fileinfo.h                          fsshell          README.md
student@student:~/csc415-filesystem-josephb333$ cat cp2l
cat: cp2l: No such file or directory
student@student:~/csc415-filesystem-josephb333$ cat cp2ltest
student@student:~/csc415-filesystem-josephb333$
```

cp2fs (empty file)

```
student@student:~/csc415-filesystem-josephb333$ touch emptycp2fstest
student@student:~/csc415-filesystem-josephb333$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
Existing filesystem detected. Loading VCB...
VCB loaded: MagicNo=0x41534442, numBlocks=19531, blockSize=512, FSM=1, RootDir=6
In loadDirectory(): lbaReading block number : 6
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    ON    |
| rm                   |    ON    |
| cp                   |    ON    |
| mv                   |    ON    |
| cp2fs                |    ON    |
| cp2l                 |    ON    |
|---------------------------------|
Prompt > cp2fs emptycp2fstest
DEBUG: Found free index at 3
In Touch(): Tail is now set to lba number: 13
DEBUG: touch success! made empty file emptycp2fstest at block 12
In b_open free fcb index found at : 0
Prompt > ls
In loadDirectory(): lbaReading block number : 6

dir1
emptycp2fstest
Prompt > cat emptycp2fstest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
Prompt >
```

Screenshots of Commands (Files w/ Contents)

cp2fs & cat

```
Prompt > cp2fs HEREISFILE
DEBUG: Found free index at 4
In Touch(): Tail is now set to lba number: 14
DEBUG: touch success! made empty file HEREISFILE at block 13
In b_open free fcb index found at : 0
Tail is now set to lba number: 17
In ensure allocation: allocated 3 blocks.
In ensure allocation: tail is now at lba 17 .
In updateFileSize on disk: 3 new blocks allocated for filename: HEREISFILE
Prompt > cat
Usage: cat srcfile
Prompt > cat HEREISIFLE
In b_open : getFileInfo failed
Failed to open file system file: HEREISIFLE
Prompt > cat HEREISFILE
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
HELLO FROM THE LINUX SIDE 661256466554381848240343575034901766360992649916334657622014980145
191238918592687339836530393887264326429951433585045690077715859869340249686694340283504163457022411806633040456823648322149407649291709884486624991429087992986642456233147947048492953047980719807501771770875381443562635226273495975672560926728096272201852685738840375462331499410484257218860173970024937710385978949352294638874287215930948390792479864689759029679908713843203529304159229725861615620844360767246237414423131395252382541214722436789521357506910806784385239131212667915286065697223577192349536631069819291852420161751071280762096700317526464632909287656212295184214611991694189593171893703770962230390480751978487698398585948551435467580934582016303889554914731649031611902973368535645741909205082336233397713399375892739362196688036541411080980862571111620497249470860494146838137541220271880030757276143464395289644876909915866493212206250053550400385293673376701537468360960764657913786708380781323834871961191069325
5294339716425075 HELLO FROM THE LINUX SIDE
Prompt >
```

mv

```
Prompt > mv HEREISFILE mvtest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
DEBUG: Found free index at 5
In Touch(): Tail is now set to lba number: 18
DEBUG: touch success! made empty file mvtest at block 17
In b_open free fcb index found at : 1
Tail is now set to lba number: 21
In ensure allocation: allocated 3 blocks.
In ensure allocation: tail is now at lba 21 .
In updateFileSize on disk: 3 new blocks allocated for filename: mvtest
File name in isFile():HEREISFILE
In fs_delete going to delete filename:HEREISFILE
In fs_delete successfully deleted: 3
writeDirToDisk: writing : 1 blocks to LBA: 6
Prompt > ls
In loadDirectory(): lbaReading block number : 6

dir1
emptycp2fstest
mvtest
Prompt > cat mvtest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
HELLO FROM THE LINUX SIDE 661256466554381848240343575034901766360992649916334657622014980145
191238918592687339836530393887264326429951433585045690077 71
585986934024968669434028350416345702241180663304045682364832214940764929170988448662499142 90
879929866424562331479470484929530
479810719807501771770875381443562635226273495975672560926728096272201852685738840375462331 49
941048425721886017397002493771038
597894935229463887428721593094839079247986468975902967990871384320352930415922972586161562 08
443607672462374144231313952523825
412147224367895213575069108067843852391312126679152860656972235771923495366310698192918524 20
161751071280762096700317526464632
909287656212295184214611991694189593171893703770962230390480751978487698398585948551435467 58
093458201630388955491473164903161
190297336853564574190920508233623339771339937589273936219668803654141108098086257111162049 72
494708604941468381375412202718800
307572761434643952896448769099158664932122062500535504003852936733767015374683609607646579 13
786708380781323834871961191069325
5294339716425075 HELLO FROM THE LINUX SIDE
Prompt >
```

cp

```
Prompt > cp mvtest cpcontents
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
DEBUG: Found free index at 4
In Touch(): Tail is now set to lba number: 22
DEBUG: touch success! made empty file cpcontents at block 21
In b_open free fcb index found at : 1
Tail is now set to lba number: 25
In ensure allocation: allocated 3 blocks.
In ensure allocation: tail is now at lba 25 .
In updateFileSize on disk: 3 new blocks allocated for filename: cpcontents
Prompt > ls
In loadDirectory(): lbaReading block number : 6

dir1
emptycp2fstest
cpcontents
mvtest
Prompt > cat cpcontents
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
HELLO FROM THE LINUX SIDE 661256466554381848240343575034901766360992649916334657622014980145
191238918592687339836530393887264326429951433585045690077 71
585986934024968669434028350416345702241180663304045682364832214940764929170988448662499142 90
879929866424562331479470484929530
479810719807501771770875381443562635226273495975672560926728096272201852685738840375462331 49
941048425721886017397002493771038
597894935229463887428721593094839079247986468975902967990871384320352930415922972586161562 08
443607672462374144231313952523825
412147224367895213575069108067843852391312126679152860656972235771923495366310698192918524 20
161751071280762096700317526464632
909287656212295184214611991694189593171893703770962230390480751978487698398585948551435467 58
093458201630388955491473164903161
190297336853564574190920508233623339771339937589273936219668803654141108098086257111162049 72
494708604941468381375412202718800
307572761434643952896448769099158664932122062500535504003852936733767015374683609607646579 13
786708380781323834871961191069325
5294339716425075 HELLO FROM THE LINUX SIDE
Prompt >
```

cp2l

```
Prompt > mv cpcontents cp2lcontentstest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
DEBUG: Found free index at 6
In Touch(): Tail is now set to lba number: 26
DEBUG: touch success! made empty file cp2lcontentstest at block 25
In b_open free fcb index found at : 1
Tail is now set to lba number: 29
In ensure allocation: allocated 3 blocks.
In ensure allocation: tail is now at lba 29 .
In updateFileSize on disk: 3 new blocks allocated for filename: cp2lcontentstest
File name in isFile():cpcontents
In fs_delete going to delete filename:cpcontents
In fs_delete successfully deleted: 3
writeDirToDisk: writing : 1 blocks to LBA: 6
Prompt > ls
In loadDirectory(): lbaReading block number : 6

dir1
emptycp2fstest
mvtest
cp2lcontentstest
Prompt > cp2l cp2lcontentstest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
Prompt >
```

```
Prompt > cp2l cp2lcontentstest
In getFileInfo() Entry populated with isDir value of :0
In b_open free fcb index found at : 0
Prompt > exit
System exiting
student@student:~/csc415-filesystem-josephb333$ ls
 b_io.c                          fileinfo.h       fsshell.c      shrek
 b_io.h                          FreeSpaceMap.c   fsshell.o      VCB.c
 b_io.o                          FreeSpaceMap.o   HEREISFILE     VCB.h
 cp2lcontentstest                fsInit.c         Hexdump        VCB.o
 cp2ltest                        fsInit.o         Makefile       vector.c
 DE.h                            fsLow.h          mfs.c          vector.h
 DirectoryEntry.c                fsLowM1.o        mfs.h          vector.o
 DirectoryEntry.o                fsLow.o          mfs.o
"Don't Worry We Don't Byte - MS1.pdf"   FSM.h     README.md
 emptycp2fstest                  fsshell          SampleVolume
student@student:~/csc415-filesystem-josephb333$ cat cp2lcontentstest
HELLO FROM THE LINUX SIDE 661256466554381848240343575034901766360992649916334657622014980145
191238918592687339836530393887264326429951433585045690077 71
585986934024968669434028350416345702241180663304045682364832214940764929170988448662499142 90
879929866424562331479470484929530
479810719807501771770875381443562635226273495975672560926728096272201852685738840375462331 49
941048425721886017397002493771038
597894935229463887428721593094839079247986468975902967990871384320352930415922972586161562 08
443607672462374144231313952523825
412147224367895213575069108067843852391312126679152860656972235771923495366310698192918524 20
161751071280762096700317526464632
909287656212295184214611991694189593171893703770962230390480751978487698398585948551435467 58
093458201630388955491473164903161
190297336853564574190920508233623339771339937589273936219668803654141108098086257111162049 72
494708604941468381375412202718800
307572761434643952896448769099158664932122062500535504003852936733767015374683609607646579 13
786708380781323834871961191069325
5294339716425075 HELLO FROM THE LINUX SIDE
student@student:~/csc415-filesystem-josephb333$
```

rm (on dir w/ contents)

```
Prompt > md fulldir
In mkdir(): Tail is now set to lba number: 30
writeDirToDisk: writing : 1 blocks to LBA: 29
writeDirToDisk: writing : 1 blocks to LBA: 6
Prompt > cd fulldir
In loadDirectory(): lbaReading block number : 29
Prompt > touch a
DEBUG: Found free index at 2
In Touch(): Tail is now set to lba number: 31
DEBUG: touch success! made empty file a at block 30
In b_open free fcb index found at : 0
Prompt > touch b
DEBUG: Found free index at 3
In Touch(): Tail is now set to lba number: 32
DEBUG: touch success! made empty file b at block 31
In b_open free fcb index found at : 0
Prompt > cp2fs shrek
DEBUG: Found free index at 4
In Touch(): Tail is now set to lba number: 33
DEBUG: touch success! made empty file shrek at block 32
In b_open free fcb index found at : 0
Tail is now set to lba number: 108
In ensure allocation: allocated 75 blocks.
In ensure allocation: tail is now at lba 108 .
In updateFileSize on disk: 75 new blocks allocated for filename: shrek
Prompt > ls
In loadDirectory(): lbaReading block number : 6
In loadDirectory(): lbaReading block number : 29

a
b
shrek
Prompt > cd ..
In loadDirectory(): lbaReading block number : 6
Prompt > ls
In loadDirectory(): lbaReading block number : 6

dir1
emptycp2fstest
fulldir
mvtest
cp2lcontentstest
Prompt > rm fulldir
DEBUG: rmdir loading dir...
In loadDirectory(): lbaReading block number : 29
DEBUG: Directory not empty, contains a at index 2
Prompt >
```